Uppsala universitet
Inst. för informatik och media

Anpassning av existerande informationsinfrastruktur med Novacura Flow

Design och integration av en kartkomponent

Jonas Gabrielson
Albin Lundberg



UPPSALA
UNIVERSITET

Kurs: Examensarbete
Nivå: C
Termin: VT-17
Datum: 170613

Sammanfattning

I denna designbaserade forskningsstudie utvecklas en kartkomponent till processhanteringsplattformen Novacura Flow. Flow erbjuder ett applikationslager som läggs ovanpå befintliga affärssystem och förmedlar ett skräddarsytt gränssnitt till slutanvändaren. Ett affärssystem kan ses som en företagstäckande informationsinfrastruktur (II) eller del därav, där Flow möjliggör anpassning till föränderliga affärsprocesser. Denna typ av anpassningsproblem är enligt Hanseth och Lyytinen något II:er ställs inför då de attraherar en tillräckligt stor användarbas. En växande användarbas ställer krav på flexibilitet för att möta oförutsedda användarkrav och användningsområden. Fallet denna designforskning kretsar kring rör anpassning av en II i ett svenskt IT-driftsbolag. Företaget är i behov av att utöka sitt digitala stöd för servicetekniker i fält genom att tillhandahålla nödvändig information i en integrerad applikation. Flow ämnar binda samman kundens affärssystem med nödvändig geografisk information vilket slutligen ger en effektivare process. Designarbetet leder till framtagandet av en prototyp vilken presenteras för kunden. Vi argumenterar för att kartkomponenten i färdigt skick kommer att förbättra kundens process, trots att den inte fullkomligt implementerats under studien. Dessutom används fallet för att undersöka Hanseth och Lyytinens designprinciper som föreslås möta olika typer av problematik som II:er ställs inför under etablering och vidare spridning. Vi ifrågasätter relevansen hos dessa designprinciper i nutida och moderna II:er och hävdar att Novacura Flow möjliggör en ny typ av anpassning av proprietära affärssystem. Vi kallar det för en "Information Infrastructure Enabler".

Nyckelord:

informationsinfrastruktur, design-baserad forskning, GIS, affärssystem, ERP, BPRS

Abstract

In this study, we develop a map component for the process management platform Novacura Flow, following a design-based research methodology. Novacura Flow provides an application layer put on top of existing business-management software, and presents a custom-tailored interface for the end user. An instance of a business-management software is a form of corporate-level information infrastructure (II), and using Flow allows for an adaptation of this infrastructure to changing business processes. This kind of adaptability problem is something II's are faced with when attracting a large enough user base, according to Hanseth and Lyytinen, as unforeseen use cases and user demands will test the flexibility of the infrastructure. The design research is centered on a case concerning the adaptation of an II in a Swedish IT maintenance company. This company needs to expand its digital support for service technicians in the field by providing necessary information in an integrated application. Flow aims to connect the customer's business-management software with relevant geographic information which will make the process more efficient. The design process leads to the development of a prototype which is presented to the customer. Even though it isn't completely done, we make the argument that this map component, when completed, will enhance the customer's process. The case is also used to examine Hanseth and Lyytinen's design principles, which are supposed to counter problems II's are exposed to during establishment and expansion. We question the relevance of these design principles in current, modern II's and propose that Novacura Flow allows for a new method of adaptation for proprietary business-management software, calling it an "Information Infrastructure Enabler".

Keywords:

information infrastructure, design science, GIS, business-management software, ERP, BPRS

Innehållsförteckning

1 Introduktion ... 1
1.1 Bakgrund ... 2
1.2 Uppdrag ... 3

2 Problemformulering ... 4
2.1 Forskningsfrågor och motivering ... 4
2.2 Avgränsningar ... 5

3 Teori ... 6
3.1 Tidigare forskning & kunskapsinventering ... 6
3.1.1 Designprinciper för informationsinfrastrukturer ... 6
3.1.2 Affärssystem ... 7
3.1.3 GIS: Geographic Information System ... 8
3.1.4 REST ... 9
3.2 Designbaserad forskning (Design Science) ... 9

4 Metodik ... 11
4.1 Forskningsmetodik inom designbaserad forskning ... 11
4.2 Metod ... 12
4.3 Datainsamlingsmetodik ... 13

5 Resultat ... 15
5.1 Problemidentifikation och motiv ... 15
5.2 Kravinsamling ... 15
5.3 Val av metod för app-till-appkommunikation ... 17
5.4 Val av teknologi för kartkomponenten ... 18
5.4.1 ESRI ArcGIS ... 18
5.4.2 MapBox och Google Maps ... 19
5.4.3 Leaflet ... 19
5.5 Utveckling av artefakt ... 19
5.5.1 Uppstartsfas och presentation av Flow-plattformen ... 19
5.5.2 Utveckling av Prototyp 1 ... 23
5.5.3 Utveckling av Prototyp 2 ... 23
5.5.4 Utveckling av Prototyp 3 ... 25
5.6 Demonstration och utvärdering ... 27

6 Analys ... 28
6.1 Kriterier för designbaserad forskning ... 28
6.2 Kunskapsprodukt ... 28
6.3 Artefakt ... 28
6.4 Forskningsfrågor ... 30
6.4.1 Har vi, genom att designa en kartkomponent, löst Kundens problem med anpassning av deras informationsinfrastruktur? ... 30
6.4.2 Kan vårt fall ge en insikt i om Hanseth och Lyytinens designprinciper bör uppdateras, och i så fall hur? ... 30

7 Slutsats ... 32

8 Diskussion 33

9 Källhänvisningar 34

1 Introduktion

En informationsinfrastruktur (II) definieras som ”en delad, förbättrande, heterogen installerad bas av IT-funktioner hos en uppsättning användargrupper som är baserad på öppna och/eller standardiserade komponenter” (Hanseth & Lyytinen, 2004, s. 1). Ett exempel på en global II är Internet i sin helhet, men II:er kan även förekomma i en (relativt) mindre skala som stödjer en sektor, en organisation eller ett företag. En II ger användare tillgång till en delad resurs som kan användas för att leverera och använda information inom gruppen. Eftersom infrastrukturen existerar för att utbyta information blir en nyckelfråga hur information från olika källor integreras med varandra. Detta kräver både system som är öppna för att utbyta information och data som lagras på ett konsekvent och likartat sätt.

Informationsinfrastrukturer började studeras under 1990-talet, och har dokumenterats genom ett flertal fallstudier som relaterat och strukturerat deras sammanhang och uppkomst (Bowker, 1996; Bygstad, 2010; Jensen, 2013; Star & Ruhleder, 1996). En grundläggande del i II-teori är att infrastrukturer ska byggas från grunden och upp genom att lokala lösningar binds ihop, istället för att definiera universella standarder först och sedan implementera dem (Ciborra & Hanseth, 1998). Till viss del har denna teori grundlagts under studiet av de så kallade Affärssystem som växte fram under 1980- och 90-talen (Broadbent & Weill, 1997).

Affärssystem är en typ av programvara eller programvarupaket som syftar till att på olika sätt underlätta och understödja företagsverksamhet, och kan därför ses som företagstäckande informationsinfrastrukturer (Corporate-wide II). En grundläggande form av affärssystem är Enterprise Resource Planning (ERP), där man använder datastöd för att på ett mer effektivt sätt planera verksamhetens resurser, men de flesta stora affärssystem innehåller idag olika moduler som exempelvis Customer Relations Manager (CRM), Management Information System (MIS), Supply Chain Management (SCM) och många fler (Zwass, 2017). Affärssystem är en miljardindustri, och från att från början huvudsakligen ha varit inriktade på stora företag har nya smidigare system lett till att även medelstora och små företag använder olika sorters datoriserat affärsstöd (Adam, Kotze, & Van der Merwe, 2011).

Att implementera ett affärssystem inom en organisation kan innebära resursbesparingar samt en ökad kvalitet hos de tjänster och produkter ett företag tillhandahåller. Medan möjligheterna är många finns även stora risker vid införandet av affärssystem. Huruvida en implementation lyckas eller ej beror till stor del på hur väl systemet matchar företaget, dess struktur, strategi och kultur (Basoglu, Daim, & Kerimoglu, 2007). Detta ställer inte bara krav på systemets möjligheter till anpassning utan kräver även organisatoriska förändringar. Många av de största aktörerna har dessutom historiskt använt stängda proprietära system för att binda sina kunder till långa avtal och även sälja supporttjänster (Crawford, 2016; Seroter, 2016). Inflexibla och komplicerade lösningar kan därför leda till stora förseningar och kostnader som dessutom riskerar att ge ett oönskat slutresultat för användaren.

1.1 Bakgrund

Novacura är ett svenskt IT-bolag huvudsakligen baserat i Göteborg och Uppsala som utvecklar en produkt de kallar Novacura Flow. Denna produkt kan ses som en processhanteringsplattform som hjälper företag att skräddarsy ett arbetsflöde för att utnyttja befintliga affärssystem på det sätt som passar deras verksamhet bäst (Novacura AB, 2015). Liknande processhanteringsplattformar finns också från andra leverantörer som Appian, Pega, Bizagi och K2[1], och går under beteckningen BPMS, Business Process Management Suites (”Business process management”, 2017; Palmer, 2014).

Genom att vara uppbyggd med ett flertal moduler kan Novacura Flow ta information från många olika databaser hos en organisation, exempelvis från existerande affärssystem från företag som SAP eller IFS, och presentera den för användaren på ett integrerat sätt i en applikation som är skräddarsydd efter affärsprocessen. Novacura Flow låter också användare skapa program eller skript grafiskt, vilket gör det lättare att utnyttja organisationens databaser, oavsett hur de är implementerade. Som ett exempel skulle ett företag som har två olika databaslösningar kunna använda Novacura Flow för att manipulera båda databaserna samtidigt genom ett gemensamt gränssnitt. Med denna funktionalitet menar Novacura att Flow kan ge en förbättrad integration och användarvänlighet som i sin tur ska ge företag större effektivitet i sina affärsprocesser. Ett av Novacuras främsta säljargument är alltså att de kan erbjuda en mer flexibel lösning på processflöden. Affärssystem är generellt sett inte byggda på ett sätt som tillåter modifikation av kunden utan att man betalar för utveckling av en egen version och/eller bryter användaravtalet vilket i sin tur inte ger vidare rätt till support. Detta innebär att kunder behöver anpassa sig, ignorera överflödiga funktioner eller gå utanför affärssystemets ramar om de vill stödja processer på ett annat sätt än vad affärssystemets utvecklare tänkt sig.

Bland Novacuras kunder finns ett av Sveriges största IT-driftsbolag som sköter om IT-infrastruktur i form av kablar, antenner och sändare runt om i landet. Tekniker i företaget använder sig av IFS applikationer för att kunna se aktuella arbetsorder i fält, logga tid och resor med mera. Detta företag (hädanefter ”Kunden”) har ett behov av att uppdatera sin affärsplattform och har valt Novacura Flow som möjliggörare. Genom verktyg som skräddarsys för kunderna med Novacura Flow kommer teknikerna enkelt få tillgång till önskad information oavsett teknisk plattform, och redigera och läsa data från flera separata databaser. Nu vill Kunden även integrera ett system för GIS (geografiskt informationssystem) i framtida versioner av deras Flow-applikation, för att tekniker enkelt ska kunna se geografiska data och detaljer om arbetsordrar, hitta rätt till installationer etc. Företaget använder redan ArcGIS som leverantör för sin kartdata, och har i detta system kartor med specifika företagsdata. Tanken är att den existerande mobila applikationen nu ska ersättas av Flow som då även kan innehålla eller referera till existerande GIS-data och ge fler möjligheter för tekniker som arbetar ute i fält. Detta kan ske på tre olika sätt:

- Genom att GIS-data kan ses och användas direkt i Novacuras mobila applikation, till exempel genom att man lägger in lämpliga API:er i den.

[1] www.appian.com; www.pega.com; www.bizagi.com; www.k2.com.

- Genom att Novacuras mobila applikation kommunicerar med en fristående GIS- applikation och på ett enkelt sätt låter fältarbetaren växla mellan de två.

- Genom att Novacura Flow hämtar GIS-data via serverside API och vidarebefordrar denna data till Novacuras mobila applikation.

Hur denna kartkomponent i slutänden implementeras påverkas till stor del av de kravspecifikationer som Kunden har. Det kan vara så att dessa krav visar på behov av olika principer för integration mellan de båda systemen. För att besluta huruvida kartkomponenten skall integreras i Flow-plattformen, utifrån de tre ovan nämnda tillvägagångssätten, behövs en sammanställning i vilken förutsättningarna kartläggs. Denna tillsammans med Kundens krav ligger sedan till grund för ett designbeslut.

1.2 Uppdrag

Vi har av Novacura fått uppdraget att hjälpa dem med utvecklingen av kartkomponenten i Novacura Flow. Uppdraget är indelat i tre moment:

- En teknisk genomgång (”review”) av olika möjligheter att lägga till denna kartkomponent till Novacura Flow.

- Möten med Kunden för att fastställa deras kravspecifikationer på en kartkomponent i Novacura Flow.

- Efter att Novacuras R&D-ansvariga valt en av lösningarna utifrån dessa kravspecifikationer skall en prototyp av kartkomponenten tas fram för demonstration.

I uppdragsbeskrivningen har Novacura föreslagit ett antal forskningsfrågor:

- Vilka förutsättningar krävs för att lyckas med app-till-appkommunikation?

- Vilka standarder existerar som man kan hålla sig till?

- Vilken teknisk lösning rekommenderas?

- Kan en systemoberoende lösning användas?

Dessa frågor illustrerar väl vad Novacura vill få ut för kunskapsprodukter ur projektet, men uppsatsen innehåller dessutom en mer teoretisk ansats. Som nämnts tidigare finns det en mer generell problematik bakom integrationen av IT-system inom informationsinfrastrukturer som detta fall kan ge insikter i.

2 Problemformulering

Som vi tidigare nämnt är affärssystem en form av företagstäckande informations-infrastrukturer, och många av de största aktörerna bygger sina system på egenutvecklade, patenterade lösningar, som exempelvis Oracle, SAP och IFS. Orsakerna till att de inte är byggda från öppna standarder kan vara många, från att det rör sig om relativt gamla system till att man vill låsa sina användare till affärssystemet. På grund av detta existerar det en grundläggande problematik kring anpassning och användande av affärssystem. Eftersom ett affärssystem är generellt utformat för att passa många olika industrisektorer och företagsformer innehåller det fler funktioner än någon enskild kund behöver. Detta i sin tur leder till utmaningar med användbarheten eftersom systemen är så omfattande. Därför blir systemets anpassningsmöjligheter avgörande för framgångsrika implementationer. Samma typ av anpassningsproblematik beskrivs av Hanseth och Lyytinen och är något II:er i allmänhet måste hantera.

2.1 Forskningsfrågor och motivering

Syftet med projektet är att hjälpa Novacura att fylla Kundens önskemål om integration mellan applikation för fältarbete och GIS. Eftersom uppsatsen utgår från en uppdragsgivare har den på ett plan behövt leverera de rapporter och resultat som företaget önskat sig från samarbetet. Detta fall relateras sedan till forskning på ett mer övergripande sätt. Hanseth och Lyytinen presenterar i sina artiklar en samling designprinciper som de menar är avgörande för en framgångsrik informationsinfrastruktur (II) (Hanseth & Lyytinen, 2010). Novacura Flow utgör tillsammans med integrerade plattformar en II såsom de beskrivs av Hanseth & Lyytinen (2004). Där nämns bland annat ett systems anpassnings- och integrationsförmåga som en utav de mest avgörande egenskaperna för en II:s fortplantning och överlevnad. Den förmågan stärks genom att hålla sig till breda standarder, utveckla kopplingar (gateways) mellan standarder och genom att designa med flexibilitet för att möta ständigt föränderliga användarkrav. Integration av GIS-information i Novacura Flow är ett typexempel på ett sådant användarkrav – funktionalitet som man inte haft i produkten tidigare, men som bör integreras för att säkerställa fortsatt expansion av infrastrukturen.

Hanseth och Lyytinen applicerar i artiklarna från 2004 och 2010 sina teorier på olika fall för att exemplifiera och legitimera sina designprinciper och regler. Fallen som används i artiklarna är ett II-projekt inom norska sjukvården samt utvecklingen av Internet. Ytterligare fallstudie har gjorts av Jensen (2013) där ett lyckat II-projekt inom dansk sjukvård presenteras. Fallet som berörs inom den här studien innebär anpassning av företagstäckande II och mer specifikt affärssystem. Om än småskaligt i jämförelse är det relevant att undersöka hur anpassningsproblematik hanteras i ett modernt fall inom en annan systemsektor. Finns det rentav andra tillvägagångssätt att hantera denna problematik än dem Hanseth och Lyytinen föreslår? Novacura Flow och annan BPMS-mjukvara erbjuder synbarligen en metod för att knyta samman och anpassa olika II, vilket alltså kan vara en alternativ lösning för organisationer vars existerande system inte integrerar och låter sig anpassas på annat sätt.

Sammantaget har denna problematik lett oss till att formulera följande forskningsfrågor:

- Kan vi, genom att designa en kartkomponent, lösa Kundens problem med anpassning av deras informationsinfrastruktur?

- Kan vårt fall ge en insikt i om Hanseth och Lyytinens designprinciper bör uppdateras, och i så fall hur?

I arbetet med att genomföra Novacuras projekt har vi skapat en **kunskapsprodukt** i form av rekommendationer och beslutsunderlag. Vårt uppdrag har också inkluderat framtagandet av en prototyp som integrerar en GIS-lösning i Novacuras system, vilket resulterar i en **artefakt**. Sammantaget har detta arbete hjälpt oss att besvara vår forskningsfråga.

2.2 Avgränsningar

Avgränsningen för uppsatsen går i linje med kraven och målsättningen Novacura har på integrationsprojektet, vilket är en balans mellan att snabbt möta kundkrav men även se till mer långsiktigt användande av komponenten. Integrationsprojektet är kundspecifikt men komponenterna tas fram med hjälp av domänstandarder vilket leder till en generell lösning som är återanvändningsbar. Uppsatsen har således begränsats till att kartlägga befintliga standarder för hantering av GIS-data, hur den kommuniceras över Internet och hur den hanteras i applikationer.

3 Teori

Avsnittet behandlar den teoretiska ramen för arbetet. Till detta hör diverse tidigare forskning och koncept, samt en genomgång av grunderna i designbaserad forskning.

3.1 Tidigare forskning & kunskapsinventering

Detta avsnitt presenterar diverse tidigare forskning och kunskap som är av nytta för förståelsen av examensarbetet. Detta inkluderar en genomgång av Hanseth och Lyytinens designprinciper för informationsinfrastrukturer, hur geografiska informationssystem fungerar, ett avsnitt om affärssystem, och en kort summering av principerna bakom REST.

3.1.1 Designprinciper för informationsinfrastrukturer

Som vi tidigare nämnt definierar Hanseth & Lyytinen (2004) en informationsinfrastruktur som ”en delad, förbättrande, heterogen installerad bas av IT-funktioner hos en uppsättning användargrupper som är baserad på öppna och/eller standardiserade komponenter”. En typ av företagstäckande II (informationsinfrastruktur) är affärssystem, som kan interagera och sträcka sig över en hel organisation. Genom bl.a. teknologiska framsteg och ökat användande utvecklas ständigt en II vilket innebär att den blir allt mer komplex. Komplexitet definierar Hanseth och Lyytinen som en dramatisk ökning i antalet av, och heterogeniteten hos, inkluderade komponenter och relationer, samt hur dessa dynamiskt och oförutsett interagerar med IT-lösningar (Hanseth & Lyytinen, 2010). Denna komplexitet menar de är något mer traditionellt sekventiella designprinciper inte tar tillräckligt stor hänsyn till. Därför föreslår Hanseth och Lyytinen ett antal designprinciper och regler vilka ämnar till att hantera dynamisk komplexitet hos II:er och därigenom säkerställa dess etablering och spridning. Dessa har sprungit ur deras studier av olika fall och har sedan observerats även i andra fall (Brænden, Hagen, & Ommundsen, 2015; Bygstad, 2010; Jensen, 2013) samt använts vid utvecklingen av nya infrastrukturer (Eriksson & Goldkuhl, 2013).

Designprinciperna Hanseth och Lyytinen föreslår adresserar två huvudsakliga problem eller utmaningar som en II står inför under dess design. Då det gäller etablering av en II handlar det om att tidigt attrahera och övertyga användare, eftersom en II saknar värde utan användare. Därför bör designen fokusera på att initialt möta krav hos en tidig användargrupp innan fullständig funktionalitet implementeras. Hanseth och Lyytinen kallar detta för ”*the bootstrap problem*”. Då användarbasen sedan växer och II:en erhåller momentum (m.a.o. växer av egen kraft, till en följd av storleken på användarbasen) ökar samtidigt komplexiteten vilket ställer krav på designen att hantera nya oförutsedda krav och användningsområden. Designen går då in i en ny fas där infrastrukturens flexibilitet sätts på prov, vilket de kallar ”*the adaptability problem*”. Designprinciperna som är ämnade att angripa dessa problem är sammanfattade i tabellen nedan.

Problem	Designprincip	Beskrivning
Designprinciper för att hantera ”*the bootstrap problem*”.	1. Designa till en början för användbarhet.	Tidiga användare löper hög risk när de adopterar nya system utan en användarbas. Därför måste designen se till deras behov och direkt skapa nytta. Vidare ska den vara enkel, billig och lätt att till sig.
	2. Utnyttja den existerande installerade basen.	II:en bör bygga på etablerade infrastrukturer, t.ex. använda standarder, samt utveckla gateways gentemot andra infrastrukturer och tjänster.
	3. Utöka den installerade basen genom att locka fler användare (strategier för övertalning) för att skapa momentum.	Användare före funktionalitet. I och med ökad komplexitet kommer användarna själva ställa krav på funktionaliteten. Organisera användargrupp och lyssna till feedback.
Designprinciper för att hantera ”*the adaptability problem*”.	4. Undvik inlåsning genom att göra infrastrukturen enkel.	Det är enklare att förändra något enkelt än något komplext. Utveckla hellre delvis överlappande delar än en ”allt-i-ett”-lösning.
	5. Gör II:en modulär genom att separera nyckelfunktioner.	Dela upp II i moduler och se till att de är löst kopplade. Använd gateways mellan olika moduler och infrastrukturer.

Tabell 1: Sammanställning av Hanseth och Lyytinens designprinciper (Hanseth & Lyytinen, 2010).

3.1.2 Affärssystem

Affärssystem eller ERP-system (enterprise resource planning) är system som koordinerar och distribuerar aktiviteter, beslut och information inom ett företag med syfte att integrera verksamhetsgrenar och därigenom effektivisera resursanvändningen (Adam m.fl., 2011). Affärssystem dök upp först på 1990-talet och ersatte dåtidens industriellt tillverkningsinriktade system. Utvecklingen har gått ifrån enkel lagerhantering till att idag innefatta områden såsom ekonomihantering, distribution, projekthantering, HR, försäljning och tack vare Internet integrera andra moderna affärsmoduler såsom Supply Chain Management (SCM), Customer Relationship Management (CRM), Business Intelligence (BI) etc. Traditionellt har affärssystem varit synonymt med stora företag. Idag ser utveckling annorlunda ut då även små och medelstora företag har börjat använda affärssystem (Adam m.fl., 2011). Skiftningen beror dels på ökade konkurrens och teknikutveckling, men även en marknadsmättnad inom storföretag vilket lett till att affärssystems-leverantörer sökt efter nya marknadsområden.

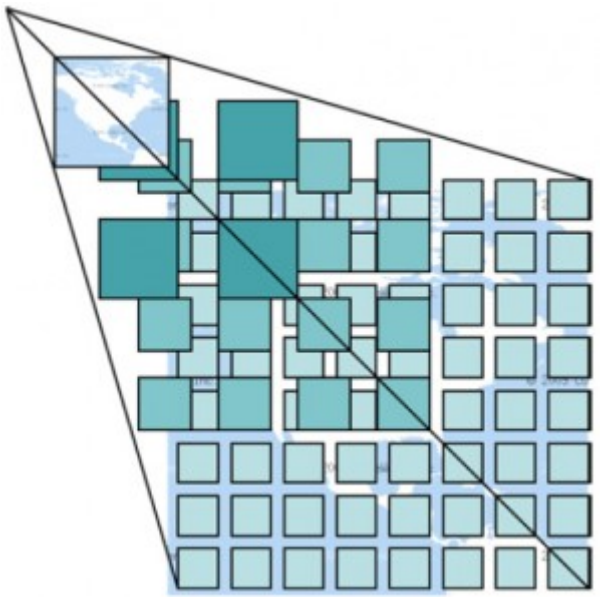
Affärssystem är en kategori mjukvaror som sällan utvecklas ”in-house” utan betraktas som ”off-the-shelf packages”. Syftet är att erbjuda generella systempaket som passar ett stort antal företagstyper och användningsområden (Brehm, Heinzl, & Markus, 2001). Trots anpassningsmöjligheter tillkommer ofta organisatoriska krav på företagen i form utav processförändringar, användarutbildning, projekthantering etc., för att nå ett lyckat slutresultat (Adam m.fl., 2011). Den mer tekniska aspekten av denna anpassningsproblematik kretsar kring konfiguration och modifikation av affärssystemet. Där konfiguration innebär anpassning till företagsprocesser genom parametrar i affärssystemet. Då mer specifik anpassning krävs kan förändring ske genom

modifikation, förändringar i kodbasen, eller genom tredjepartsmjukvara som körs ovanpå existerande affärssystem.

3.1.3 GIS: Geographic Information System

Geografiska informationssystem (GIS) är system för att skapa, lagra, och analysera kartografiska data (DeMers, 2017). Ursprungligen gjordes denna form av kartanalys manuellt men sedan de första datoriserade kartorna skapades på 50- och 60-talen har nu digitala kartor blivit allt viktigare eftersom de möjliggör snabbare och mer avancerade analyser.

Generellt består GIS av flera olika lager, som kan vara antingen vektor/objekt eller rasterbaserade. I botten ligger normalt ett ”*Basemap Layer*”, det lager som utgör den egentliga kartan. Denna baskarta består av bilder, som utgör kartans olika zoomnivåer. För varje nivå man zoomar in i kartan från nivå 0, som är den yttersta nivån, fördubblar man upplösningen horisontellt och vertikalt. Eftersom detta resulterar i mycket stora bilder när man zoomat in några nivåer i kartan är varje kartnivå uppstyckad i kvadrater kallade tiles. Varje sådan tile har samma pixelstorlek, och varje nivå fyrdubblar antalet tiles (Mapbox, Inc., 2017a). Till exempel; vid zoomnivå 13, vilket ungefär motsvarar att ett par tiles kan täcka en medelstor stad, kommer hela världskartan bestå av totalt över 67 miljoner tiles (4^{13}). Google Maps använder 360 miljarder 256 x 256 pixlar stora tiles för att täcka hela världen med 20 zoomnivåer, från en världstäckande tile ner till en skala om ungefär 1:1100 (Miller, 2010). De flesta GIS-leverantörer tillhandahåller flera olika baskartor, som satellit, vägnät, natur och olika färgstilar. Tidigare var varje sådan baskarta rasterbaserad, men förutom satellitkartor är numera de flesta baskartor vektorbaserade, vilket gör dem mer utrymmeseffektiva och möjliggör snabbare laddning och hantering.



Figur 1: Schematisk illustration av zoomnivåer och tiles i ett Basemap Layer.

Ovanpå baskartan kan man lägga ”*Feature Layers*”, vilket är informationslager med punkter och polygoner. Dessa lagras och överförs i något standardiserat format, och visualiseras som punkter, streck eller ytor av klienten. Dessa objekt kan sedan innehålla valfria data som kan hämtas ut och representeras på olika sätt beroende på klienten. En vanlig modell är att den

geografiska punkten eller ytan representeras på kartan med en markör eller ett fält, och att man kan se data som hör till objektet när man klickar på det, exempelvis i ett sidofält eller en text-bubbla (”popup”).

En vanlig standard för att överföra data om geografiska objekt är GeoJSON, en variant av JSON (JavaScript Object Notation) med en bestämd syntax för att passa detta syfte. Utöver den vanliga koordinatinformationen kan ett GeoJSON-objekt innehålla valfri mängd ytterligare data. I och med att det är baserat på standardformatet JSON kan t.ex. en JavaScript-applikation enkelt hantera datan utan konvertering (Bray, 2014).

3.1.4 REST

REST, vilket står för Representational State Transfer, är egentligen ingen standard utan en arkitekturstil (Fielding & Taylor, 2000). Däremot använder REST-tjänster flera standarder, som t.ex. HTTP, URL, HTML, XML etc. Tanken bakom REST är att ta hänsyn till och utnyttja redan väletablerade protokoll och designa webbtjänster efter protokollets restriktioner. Det dominerande protokollet för webben är HTTP-protokollet. Webben är i sin tur uppbyggd av resurser som nås via dess URL-adress. Med hjälp av HTTP-förfrågningar kan sedan resurser hämtas (GET), skapas (POST), tas bort (DELETE) etc. Det är just dessa standardiserade förfrågningar REST bygger på, vilket möjliggör en plattformsoberoende kommunikation.

I och med att REST-tjänster som nämnts bygger på HTTP-protokollet så är också klient-förfrågan-server förhållandet detsamma. En klient frågar efter en resurs och servern svarar med en representation av den resursen vilket försätter klienten i ett nytt tillstånd (state). I vanliga fall då man navigerar på webben är den representationen en HTML-fil som webbläsaren tolkar. Hur förfrågningen behandlas av servern känner aldrig klienten till. På samma sätt fungerar REST-tjänster. Men representationen kan vara en annan, exempelvis i form av en XML-fil, JSON eller dylikt. Implementeringen på serversidan är alltså oväsentlig och kan förändras eller uppdateras utan att påverka klienterna. Detta kallas för *loose coupling*, löst kopplade.

3.2 Designbaserad forskning (Design Science)

Grunden för den designbaserade forskningen (Design Science) lades under 1960-talet av arkitekten Buckminster Fuller, som definierade det som ett systematiskt sätt att utföra design på (Fuller, u.å.; Fuller & McHale, 1964). Denna idé populariserades i slutet av 60-talet av Herbert Simon som i sin bok *The Sciences of the Artificial* (Simon, 1969) definierade begreppet design science som det vetenskapliga studiet av design. Genom åren har dessa två definitioner omformulerats och förändrats allt eftersom designbaserad forskning har börjat användas inom andra discipliner än arkitektur, t.ex. ingenjörsvetenskap, stadsplanering, medicin, verksamhetsstyrning och informationssystem.

Inom området informationssystem (IS) har Hevner m.fl. (2004) lagt fram sju riktlinjer för att stödja användningen av designbaserad forskning inom IS. I korthet sammanfattar de sina riktlinjer så här:

1. **Design as an Artifact**: Designbaserad forskning måste producera en artefakt av någon form, som en konstrukt, en modell, en metod eller en instans.

2. **Problem Relevance**: Målet med designbaserad forskning är att skapa teknikbaserade lösningar till viktiga och relevanta affärsproblem.

3. **Design Evaluation**: Designartefaktens användbarhet, kvalitet och effektivitet måste demonstreras med väl utförda utvärderingsmetoder.

4. **Research Contribution**: Bra designbaserad forskning måste ge klara och verifierbara bidrag till kunskap om designartefakten, designgrunderna och/eller designmetoderna.

5. **Research Rigor**: Designbaserad forskning bygger på att man använder rigorösa metoder vid både konstruktionen och utvärderingen av artefakten.

6. **Design as a Search Process**: Att finna en effektiv artefakt kräver att man använder de medel som finns till hands för att nå det önskade målet medan man observerar vad som gäller i problemets omgivning.

7. **Communication of Research**: Designbaserad forskning måste presenteras effektivt både för en teknikorienterad och en verksamhetsorienterad publik.

Dessa sju riktlinjer utgör en teoretisk grund för att utföra, utvärdera och presentera designbaserad forskning.

4 Metodik

Avsnittet presenterar den metodik som använts under examensarbetet, både var rör den designbaserade forskningen, hur denna metodik har omsatts i praktiken och hur data har samlats in.

4.1 Forskningsmetodik inom designbaserad forskning

Designbaserad forskning är relativt nytt inom informationssystem (IS). Majoriteten av den forskning som produceras och publiceras är beskrivande forskning, som härstammar från samhälls- och naturvetenskapliga discipliner. Här skiljer sig designbaserad forskning genom att målet är att ta fram en explicit och direkt applicerbar lösning på ett definierat problem. Denna typ av forskning är utbredd inom t.ex. ingenjörskonsten, men har ännu inte fått samma fäste inom IS. Peffers m.fl. (2007) har i sin artikel identifierat två huvudsakliga anledningar till detta, där dels områdets ringa ålder spelar in, samt det faktum att det har saknats en gemensam forskningsmetodik vilket lett till att designbaserad forskning har haft svårt att vinna acceptans och erkännande inom IS-området. Med hjälp av en gemensam metodik kan utförande, presentation och utvärdering av designbaserad forskning göras på ett mer effektivt och enhetligt sätt. I sin artikel har Peffers m.fl. därför arbetat fram en metodologi som sedermera har använts brett och även är förenlig med tidigare litteratur.

Denna forskningsprocess är uppdelad i sex steg (Peffers m.fl., 2007):

1. **Identify problem & motivate**: Definiera ett specifikt forskningsproblem och rättfärdiga värdet av lösningen. Detta fungerar sedan som ett motiv för forskaren och andra intressenter i utvecklingen av en artefakt.

2. **Define objectives of a solution**: Definiera målsättningar med lösningen utifrån problemspecifikationen. Ta reda på hur den nuvarande lösningen ser ut och fastställ krav för den nya lösningen.

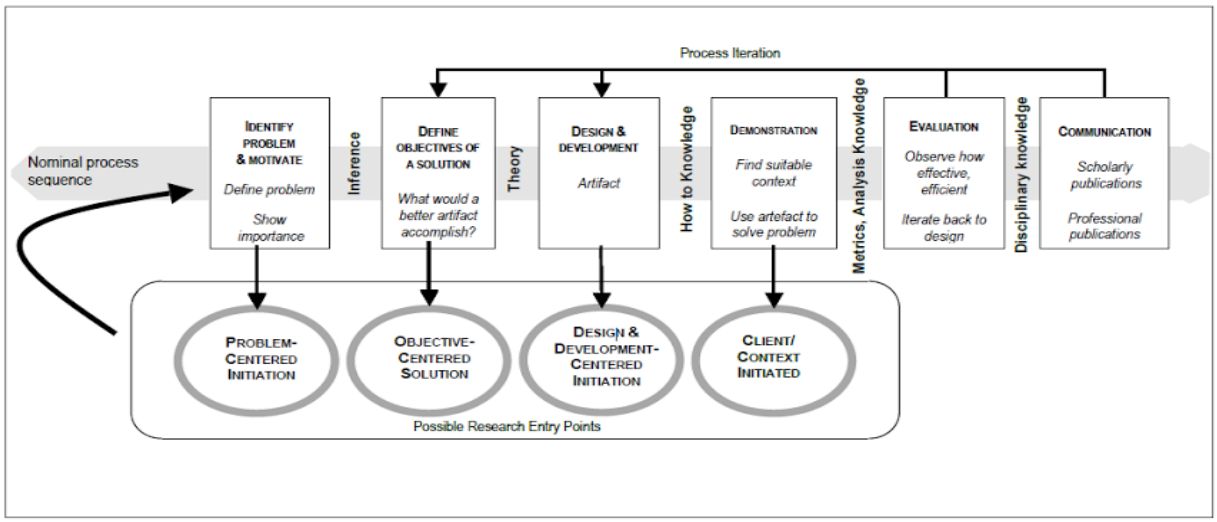
3. **Design & development**: Skapandet av själva artefakten som antingen är av typen konstrukt, modell, metod eller instans.

4. **Demonstration**: Demonstration av artefaktens förmåga att lösa det initiala problemet.

5. **Evaluation**: Utvärdera artefaktens förmåga att lösa problemet. Med hjälp av lämplig mätmetod avgörs om artefakten har nått upp till målsättningarna. Forskaren avgör därefter om resultatet är tillfredsställande. Om inte så iterera tillbaka till steg tre och försök förbättra designen.

6. **Communication**: Ändamålsenlig kommunikation av arbetet utåt. Problemets relevans och artefaktens användbarhet påvisas för forskare och andra intressenter.

Forskningsprocessen har sammanfattats grafiskt av Peffer m.fl. (2007) i Figur 2. Processen presenteras här sekventiellt men behöver inte nödvändigtvis följas från början till slut. Som figuren illustrerar kan forskningen initieras i ett annat skede än vid problemidentifikationen. Arbetet som utförs inom denna uppsats initieras t.ex. i processteg två; ett av företag identifierat problem vilket kan lösas med hjälp av en IT-artefakt.



Figur 2: Design science research methodology (DSRM) (Peffers m.fl., 2007).

4.2 Metod

Fokus i detta arbete har lagts på de inledande stegen i den designbaserade forskningsmetod som Peffer m.fl. (2007) beskriver, och de steg som hamnar utanför ramarna hanteras genom en hypotetisk diskussion om hur dessa steg skulle kunna se ut. I denna sektion beskrivs den arbetsgång som följts under arbetet.

Identify problem and motivate: Ursprungsproblemet identifierades av Kunden redan innan uppdragets början. Detta problem och dess motiv delgavs muntligen i detalj vid ett initialt möte. Kunden hade redan vid det tillfället arrangerat en workshop med berörda medarbetare som resulterat i en oprioriterad kravlista.

Define objectives of a solution: För att utröna målet med projektet undersöks existerande kravspecifikationer från Kunden, och de listade kraven rangordnas efter prioritet. Dessa prioriteringar görs i visst samråd med Kunden men är i huvudsak en guide för vad som ska uppnås under utvecklingsfasen för att nå en godtagbar produkt enligt utvärderingssteget.

Design and development: Novacura har en utvecklingsavdelning i Göteborg som ansvarar för utvecklingen av deras produkt Novacura Flow. Tillsammans med dem avgörs under ett flertal möten hur kartintegrationen ska utföras. Olika tekniska möjligheter och hinder för integrationen undersöks, under löpande kontakt med ansvariga hos Kunden och projektledare på Novacura.

En tidig prototyp utvecklas för att demonstrera hur en integration kan komma att se ut hos klienterna. Flow har applikationer för många olika plattformar men en första prototyp tas fram som en JavaScript-baserad webbapplikation. Detta utförs med hjälp av open source-kartor och GIS-standarder. Målet med en tidig och enkel prototyp är att snabbt kunna presentera denna för Kunden, därigenom inhämta feedback från användare och sedan steg för steg lägga till ny funktionalitet i efterkommande versioner (iterativ utveckling).

Demonstration: Det som presenteras gentemot Kunden är ett ”proof of concept” som ska visa hur kartmodulen i Flow kan komma att se ut när den är färdigutvecklad. Kartintegrationen innefattar även utveckling av en GIS-konnektor för att kunna demonstreras mot ett riktigt dataset i en GIS-databas. Omfånget av uppsatsen medför att det endast finns möjlighet att presentera en tidig prototyp.

Evaluation: Efter demonstration utvärderas prototypen gentemot den existerande kravspecifikationen och det bedöms om den kan nå upp till en nivå som motsvarar en ”minsta brukbara produkt” (Minimal Viable Product, MVP). Med detta avses att den har tillräcklig funktionalitet och stabilitet för att fungera även om den bara uppfyller en del av de uppsatta kraven. Utvärdering av en artefakt kan ta många olika former – observationer, analyser, experiment, tester eller beskrivning (Hevner m.fl., 2004, s. 86) – och i detta fall sker utvärderingen alltså genom en kvalitativ, kognitiv beskrivning av hur väl den existerande prototypen kan antas fungera i fält, utgående från hur väl den uppfyller kravspecifikationen.

Communication: För att fastställa designarbetet med kartmodulen som ett relevant exempel på designvetenskap dokumenteras uppdraget noggrant och presenteras i form av en uppsats.

4.3 Datainsamlingsmetodik

För att skapa vår kunskapsprodukt och i förlängningen vår föreslagna artefakt har vi använt oss av en iterativ undersökningsprocess där vi gått igenom olika alternativ för design av artefakten, för att sedan låta Novacuras utvecklingsavdelning ta ställning till vilken väg som är lämpligast för dem att fortsätta på.

Vår uppdragsbeskrivning har baserats på möten med både Novacura och Kunden, vilket sedan också lett till vår problembeskrivning. Dessa möten har skett kontinuerligt under utvecklingsprocessen, och vi har i och med detta också förändrat problembeskrivning efter hand som vi fått ytterligare information om de existerande systemen. Eftersom detta huvudsakligen har varit kundmöten har vi inte använt någon formell intervjumetodik under mötena, utan hållit mer informella samtal vars innehåll vi har antecknat under tiden på papper och dator. Ett av de huvudsakliga syftena med dessa möten har varit att fastställa och prioritera Kundens kravspecifikation, och därför anser vi det vara motiverat att inte hantera detta som en formell undersökande intervju med inspelning och transkribering. Istället har vi förberett varje möte med ett antal frågeställningar som vi behövt få besvarade för att kunna fortsätta vårt arbete med uppdraget, och styrt samtalet för att få de svar vi känt att vi behövt. Detta skulle kunna liknas vid en form av semistrukturerad intervju, även om dessa möten till formen inte var intervjuer utan öppna samtal.

Under utvecklingsarbetet av prototypen/artefakten har vi behövt lära oss mer både om specifika API:er för olika geografiska informationssystem och om de programmeringsspråk och utvecklingsmiljöer som de använder. Vi har alltså börjat med att skapa en förförståelse av hur man kan implementera de önskade kartfunktionerna på olika sätt och sedan testat oss fram för att undersöka de olika systemens styrkor och svagheter. Detta har vi gjort i en undersökande, iterativ process genom att gå igenom dokumentation och göra praktiska experiment. Vi har dessutom fått ta del av Novacuras kodbas och satt oss in i hur deras applikation fungerar för att kunna programmera den nya kartkomponenten. Som stöd för denna datainsamling har vi kontinuerligt fört loggbok över hur vi arbetat och vad vi utfört, och den kod vi skrivit har vi kommenterat för

framtida analys. Vi har arbetat agilt under utvecklingen, bland annat med hjälp av verktyget Trello[2], som vi använt som en virtuell Kanbantavla av den typ som används vid Scrum/Lean-utveckling (Al-Baik & Miller, 2015).

[2] www.trello.com.

5 Resultat

I denna sektion presenterar vi resultaten av vår designforskning. Presentationen är baserad på de sex steg som beskrivs i Peffers m.fl. (2007), men vi har valt att anpassa den efter vårt specifika fall.

5.1 Problemidentifikation och motiv

Forskningen baseras på ett redan väl identifierat problem inom ett svenskt IT-driftsbolag (”Kunden”) där nya tekniska lösningar krävs för att effektivisera deras processer. Kunden verkar över stor geografisk yta där de har ägande och serviceansvar för en signifikant del av det svenska telekommunikationsnätet. Behovet har uppstått då koordinering av Kundens servicepersonal inte fungerar tillfredsställande. Koordinationen av servicepersonal görs via en central enhet och varje servicetekniker har sedan en mobil enhet för att ta emot arbetsordrar (AO). Kunden använder sig av flera system som stödjer denna verksamhet. Systemen har bristande interoperabillitet och saknar ett integrerat gränssnitt gentemot serviceteknikern. Ett vanligt scenario är att driftcentralen får ett larm om en trasig installation. En ledig tekniker tilldelas AO och utför arbetet varefter hen sedan är ledig för ny tilldelning. Här används idag en lösning som kan effektiviseras. I stället för att jobba utefter den faktiska processen anpassas arbetet efter nuvarande applikationsmall som inte är anpassad efter användaren.

Kunden använde vid utgångsläget ett flertal olika informationskällor i sitt underhålls- och reparationsarbete. Teknikerna tilldelas arbetsordrar från en central enhet men har inte själva tillgång till all information då den lagras i olika system. Väl ute i fält finns ingen enhetlig lösning för att geografiskt visualisera data och servicetekniker använder olika verktyg för att hitta installationer. Med hjälp av Novacura Flow vill Kunden ta fram processer som sedan gestaltas som ett arbetsflöde i en mobil enhet, där varje steg i arbetet sköts inom samma applikation. Detta medför en sömlös process utan avbrott med integrerad och processpecifik information.

Novacura Flow som affärsplattform har genom ökat användande, inom allt fler branscher, ställts inför nya kundkrav. Systemet används därigenom på nya och oförutsedda sätt. En lätthanterlig konfiguration av arbetsflöden och applikationer är Flows huvudsyfte där ett tätt samarbete med användarna är centralt. Kartintegrationen där arbetsordrar (AO) eller andra objekt presenteras geografiskt istället för i en lista ses av företaget som ytterligare ett steg för att effektivisera arbetet för dess användare och möta nya kundbehov.

5.2 Kravinsamling

Med utgångspunkt i de problem som identifierats i den existerande lösningen hade Kunden innan ärendet kom till Novacura hållit i en workshop med relevant personal i organisationen för att reda ut önskemål och krav på en nyutvecklad lösning. En sammanfattning av kravspecifikationen på kartfunktionen kan ses i Tabell 2 nedan.

Kundens användarkrav	Vår prioritet	Vår kommentar
Man ska kunna se på en karta var en Arbetsorder (AO) ska utföras.	Hög	Grundläggande funktion.
Man ska lätt kunna se vad AO:n har för prioritet och vad det är för typ av AO.	Hög	Bör implementeras med färg på markörer och popups.
Man ska kunna se vilken kund det är.	Medel	Kommer stå i AO som hämtas från affärssystemet IFS.
Man ska se om det finns särskilda krav på AO:n, om specifika kompetenser behövs och hur många tekniker som behövs.	Låg	Kommer stå i AO, oklart om det måste visas direkt på kartan på något vis.
Det ska gå att se hur man kommer in på platsen (Access).	Medel	Kommer stå i AO.
Det ska finnas en kort beskrivning av ärendet.	Medel	Kommer stå i AO.
AO:ns status ska gå att se och en notis ska sändas om det ändras från något håll.	Låg	Status kommer kunna ses, men hur förändringar ska propageras är oklart.
Det ska framgå om AO:n kräver reservdelar och i så fall var det närmaste lagret är som har dessa delar.	Låg	Detta kräver troligen ytterligare databaskopplingar som vi inte vet hur de ska genomföras än.
Det ska gå att filtrera efter kompetens, kund och liknande egenskaper hos AO.	Låg	Filterfunktion kommer troligen utvecklas till en senare version.
Man ska kunna se vilken Servicegrupp AO:n tillhör och vem som är områdesansvarig.	Medel	Bör gå att se i AO.
Man ska kunna zooma på kartan.	Hög	Grundläggande funktion.
Kartfunktionen ska ha en Gantt-vy som filtreras geografiskt efter vyn, och det ska gå att tilldela AO genom denna vy.	Låg	Gantt kräver en ytterligare vy som inte finns än i Novacura Flow och måste utvecklas.
Kartan ska vara länkad till IFS så man kan se närmare detaljer om en AO via en knapp till och från IFS.	Medel	Denna information ska hämtas av Flow och presenteras utan att man lämnar applikationen.
Kartan ska automatiskt uppdateras när nya AO läggs till i IFS.	Låg	Dynamisk uppdatering får vänta till senare versioner.
Kartan ska visa om det föreligger kraftigt väder som stör verksamheten i ett område, typ åska eller storm.	Låg	En rudimentär vädertjänst kan troligen implementeras, men för en bättre variant krävs sannolikt ett avtal med ex.vis SMHI.
Kartan ska visa om AO ligger i ett farligt område, som t.ex. ett militärt övningsfält.	Medel	Lite beroende på hur Kundens kartdata ser ut kommer det till en början gå att visa specifika kartlager som kan innehålla information om farliga områden eller liknande.
Vid flera AO på samma plats ska man kunna se hur många det är.	Medel	En enkel klustrering av AO går att få men kan eventuellt leda till andra problem som att man inte omedelbart ser färg eller ikon på AO-markörer.
Det ska vara enkelt att tilldela AO samt göra statusändringar vid hantering av en AO.	Låg	En viktig funktion som dock kommer behöva vidareutveckling av andra delar av Novacura Flow först.

Tabell 2: Kundens användarkrav med kommentarer, hämtade ur anteckningar från Kundens interna workshop (1/12 2016).

Eftersom dessa användarkrav uppkom under ett brainstorming-möte representerar de såväl grundläggande krav som mer lösa önskningar på framtida funktioner. Vi har prioriterat dessa krav i tre nivåer – Låg, Medel och Hög – utgående från vad vi anser vara de mest basala funkt-

ionerna och vad som kan vara lämpligt att åstadkomma för en första prototyp. I Tabell 2 presenteras dessa prioriteringsnivåer samt en kort kommentar om hur vi resonerat i denna prioriteringsordning.

Utöver dessa användarkrav från Kunden har även Novacura egna, delvis tekniska, krav på kartmodulen vilka sammanfattas i Tabell 3.

Novacuras tekniska krav	Vår prioritet	Vår kommentar
Kartfunktionen bör bygga på öppna standarder utan licenskrav	Hög	Även om Kunden använder sig av ArcGIS vill Novacura ha en licensoberoende lösning som andra kunder utan ArcGIS-licens kan använda sig av
Systemoberoende lösning	Medel	Vi bygger kartfunktionen på en webbplattform för att den ska kunna användas på alla system
Inbyggda kartfunktioner för iOS och Android	Låg	Överlåter detta till Novacuras R&D-avdelning

Tabell 3: Novacuras tekniska krav.

Som med Kundens kravlista har vi även här försett varje krav med en prioritet och en kommentar som förklarar resonemanget bakom prioritetsnivån.

Sammantaget har vi markerat fyra krav med Hög prioritet: (1) Att man ska kunna se på en karta var en arbetsorder ska utföras, (2) Att man lätt ska kunna se vad en arbetsorder är av för typ och vad den har för prioritet, (3) Att man ska kunna zooma på kartan och (4) Att kartfunktionen bör bygga på öppna standarder utan licenskrav. Dessa krav är grundläggande för kundnytta i kartmodulen, och måste finnas med i vår prototyp. Ett antal krav har markerats som Medelprioriterade – dessa är också viktiga för kundnyttan och bör finnas med i vår prototyp, men kan tvingas vänta till en senare iteration om problem uppstår vid implementeringen. Krav med Låg prioritet kan förvisso vara nog så viktiga för kunden i förekommande fall, men för flertalet av dessa krävs utveckling av ytterligare komponenter och processer inom Novacura Flow för att de ska kunna realiseras, vilket innebär att de får vänta till en senare version.

Novacura Flow saknar idag stöd för hantering av GIS information – dels genom avsaknad av en sådan *konnektor*, alltså Flows informationsflöden (gateways) mot system med GIS-information, samt avsaknad av en komponent för att visa denna information visuellt i klienten. Målsättningen är således att utveckla dessa delar för att hantera GIS-data och kunna representera detta visuellt i Flow klienten. På ett mer övergripande plan medför detta dels att Kunden kan utöka användandet av Flow och helt eller delvis avveckla den andra applikationen, dels att Flow kan användas i fler verksamheter där användning av GIS-data är ett krav.

5.3 Val av metod för app-till-appkommunikation

Kundens existerande systemlösning har medfört att deras tekniker i fält använder sig av flera mobila applikationer för att utföra sitt arbete. Först använder de sig av IFS applikation för att se sina tilldelade arbetsordrar och även för att ändra status i dem, vilket oftast kräver att de navigerar genom flera sidor av applikationen. Vill de sedan hitta till installationen som ska åtgärdas behöver de därefter använda en egen lösning i form av kart-applikationer eller GPS-mottagare. Vid vissa arbeten behöver man komma åt information i tre olika sidor i IFS, ett

Excelark och en separat databas över företagets utrustning. Att komma ifrån denna komplicerade arbetsgång är en av orsakerna till att Kunden bett Novacura att hitta en bättre lösning. I deras inledande förfrågan förutsätts det att man även i fortsättningen skulle behöva växla applikationer på något sätt, vilket också reflekteras i Novacuras ursprungliga förslag till forskningsfrågor, men efter att ha mött både Novacura och Kunden blev det uppenbart att det skulle vara att föredra för användarna om man kunde samla en komplett lösning i en och samma applikation. Av den anledningen lade vi vår undersökning av app-till-appkommunikation i de två största mobila operativsystemen iOS och Android åt sidan för att istället koncentrera oss på att samla funktionerna i Novacuras applikation. Eftersom deras tjänst till stor del bygger på en serverside-lösning blev det också naturligt att fokusera på att göra det möjligt för serverkomponenterna i Novacura Flow att kommunicera med Kundens kartleverantör, snarare än att bädda in API:er för detta i Novacuras olika applikationer på mobila enheter.

5.4 Val av teknologi för kartkomponenten

Inför framtagandet av prototypen har olika kartbibliotek, API:er (Application Programming Interface) och SDK:er (Software Development Kit) undersökts. Målsättningen har varit att utveckla en lösning som stödjer de användarkrav som ställs på kartmodulen men samtidigt inte kräver någon licens. Nedan följer en sammanfattning av de möjliga lösningar vi har undersökt.

5.4.1 ESRI ArcGIS

Kunden använder som nämnt ArcGIS-plattformen från ESRI för att hantera sin GIS-data. Plattformen innehåller i sig flera delar såsom desktopprogram, onlineplattform, serverprogram, applikationer etc. I grunden ligger ArcGIS Desktop som möjliggör bl.a. skapande, administration och analys av GIS-data. Dessa kartor kan sedan publiceras via ArcServer genom olika servertjänster som tillhandahåller t.ex. tiles för baskartor, features för geografiska objekt etc. ArcGIS tillhandahåller även en mängd applikationer för olika ändamål och därtill utvecklarverktyg såsom API:er och SDK:er för olika plattformar.

Novacuras applikationer stödjer flera olika plattformar, men själva applikationerna som körs på de olika klienterna är utvecklade med AngularJS, en form av JavaScript. Vi undersökte därför olika JavaScript API:er och bibliotek från olika företag och organisationer, däribland ArcGIS. ArcGIS var den första plattformen vi undersökte dels pga att Kunden använder dem, samt att de är den största GIS leverantören i världen (Alban, 2015). Därför blev det en naturlig utgångspunkt för våra undersökningar. ArcGIS JavaScript API är rigoröst dokumenterat och erbjuder stor funktionalitet. Däremot är kostnadsbilden svåröverskådlig och kraven som ställts på modulen är att den ska vara en öppen och generell lösning, vilket ledde till att vi inte valde ArcGIS JavaScript API för utveckling av applikationen.

Den andra delen av kartmodulen består av en konnektor, genom vilken Flow servern kommunicerar med GIS-servrar. Även konnektorn utvecklas i första hand gentemot Kunden och ArcGIS men har likadana krav på generisk funktionalitet som applikationen för att möta framtida implementeringar av Flow. ArcServer kan exponera olika GIS-tjänster genom deras REST API. Motivet bakom valet att utveckla en REST-konnektor för Flow är flexibiliteten. REST-konnektorn utvecklas inte enbart mot ArcGIS utan kan användas generellt emot andra REST API:er.

5.4.2 MapBox och Google Maps

MapBox och Google Maps är två andra kartleverantörer vi har undersökt. Båda erbjuder väldokumenterade utvecklarmiljöer och JavaScript API:er. De tillhandahåller även egna kartor, navigering och vissa analysverktyg. Likt Esri är även Google Maps och MapBox utvecklarmiljöer kostnadsfria. För att använda dessa inbäddade kartor kommersiellt krävs dock en licens (Google Inc., 2017; Mapbox, Inc., 2017b). Även om det skulle kunna vara av intresse för Novacura att använda någon av dessa lösningar kräver det ett mer affärsmässigt beslut med kostnadsanalys, varför vi lade dessa båda leverantörer åt sidan.

5.4.3 Leaflet

Leaflet är ett open-source JavaScript bibliotek som används av bland annat Flickr, Pinterest och Evernote. Den stora skillnaden jämfört med de andra undersökta kartleverantörerna är att Leaflet endast är ett JavaScript-bibliotek och därmed inte tillhandahåller egna karttjänster såsom kartor eller navigering. Leaflet erbjuder inte samma avancerade funktioner som de andra lösningarna vi undersökt, men är i gengäld avskalat och enkelt att implementera. Leaflet kräver heller ingen licens för att användas kommersiellt, men då de saknar egna kartor krävs att dessa hämtas från tredje part. Detta adderar ytterligare flexibilitet till applikationen då användaren fritt kan välja leverantör av karta. Trots att Leaflet inte erbjuder lika stor funktionalitet som de andra undersökta alternativen finns det gott om plugins som utvecklats av open-source gemenskapen (Leaflet, 2017). Detta har medfört att vi med Leaflet har kunnat utveckla en prototyp som både möter Novacuras krav på fri licens samt de användarkrav som ställts av Kunden.

5.5 Utveckling av artefakt

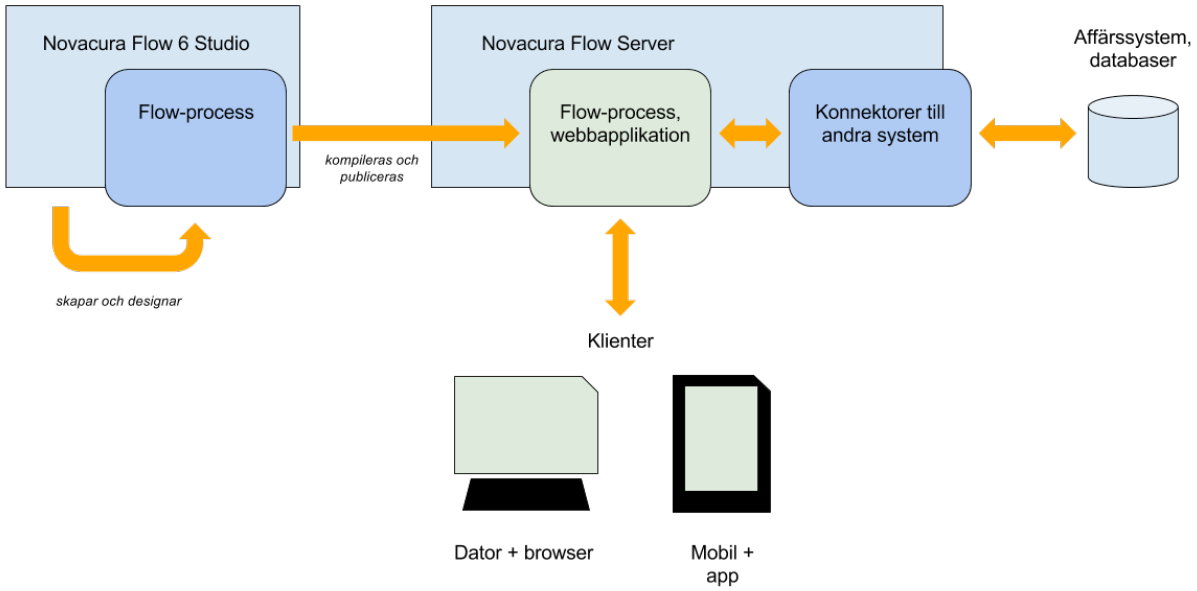
I denna sektion presenteras utvecklingen kronologiskt, som den utfördes. Varje iteration av utvecklingen behandlas i ett eget avsnitt, som avslutas med en kort sammanställning av de resultat eller lärdomar som nåddes under iterationen.

5.5.1 Uppstartsfas och presentation av Flow-plattformen

Vi påbörjade utvecklingen av artefakten i samband med att vi utvärderade de olika kartleverantörerna. Vi gick igenom utbildningsmaterial och dokumentation för de fyra leverantörer som nämnts ovan – ESRI ArcGIS, Leaflet, Mapbox och Google Maps – och skapade med hjälp av detta diverse testkartor för att skaffa oss en bild av hur de olika systemen fungerade i praktiken. Alla fyra testades i webbutveckling med Javascript, och ArcGIS testades också i C#.

Den 25:e april träffade vi delar av Novacuras R&D-avdelning på deras kontor i Göteborg, där vi efter en avstämning rörande hur kartkomponenten ska fungera fick tillgång till källkoden för Novacura Flow och en introduktion till hur Flow fungerar. Med hjälp från R&D kunde vi sedan ta fram ramverket till det som skulle bli kartkomponenten. Novacura Flow är skrivet i C# och AngularJS (ett stort Javascript-bibliotek med MVC-struktur som utvecklats av Google) och utvecklat i Microsoft Visual Studio. Även de mobila applikationerna för iOS och Android är

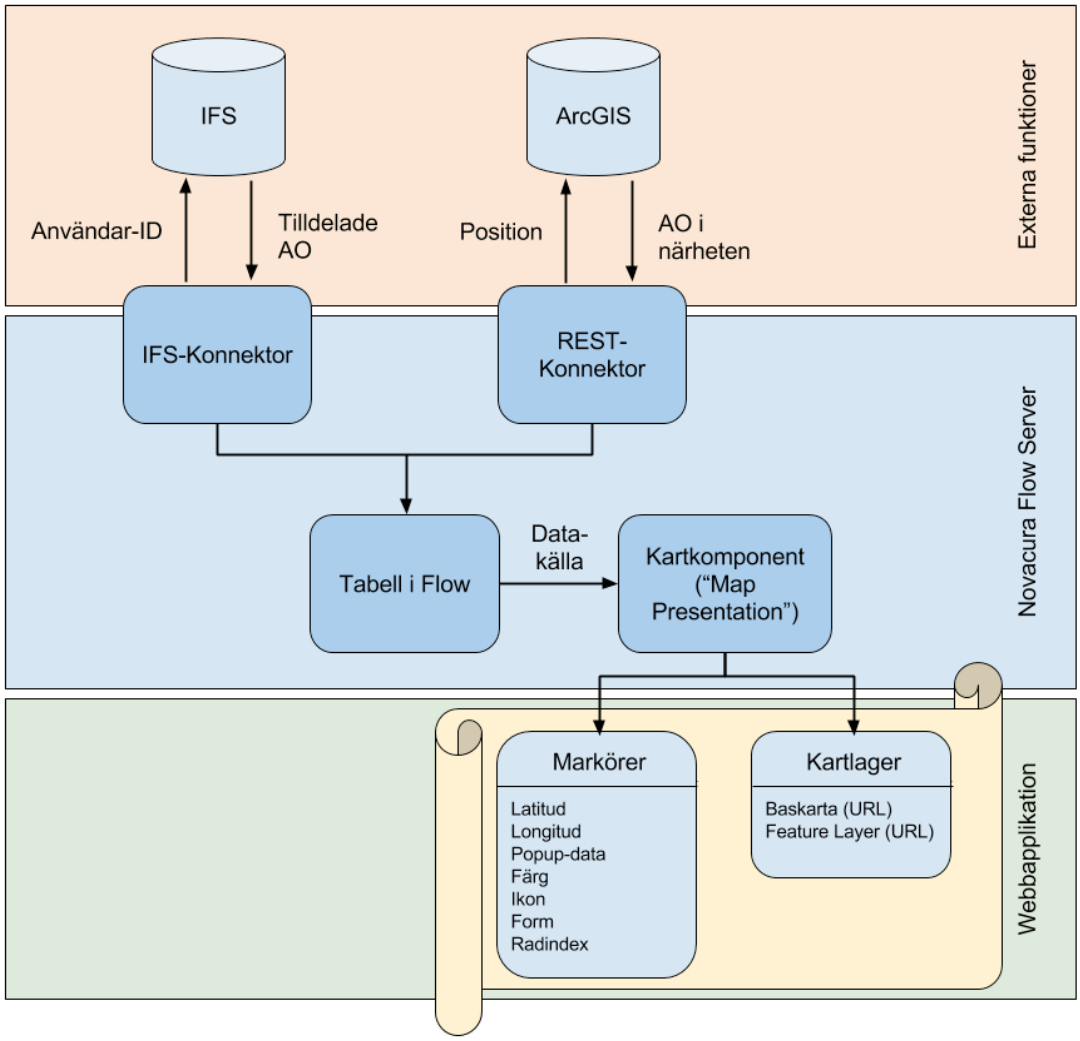
skrivna i C# och portas sedan med hjälp av Xamarin till plattformsanpassade (”native”) applikationer. Efter mötet hade vi en bättre förståelse för Flow-systemets arkitektur och hur vi skulle gå vidare med skapandet av artefakten.



Figur 3: Schematisk representation av Novacura Flow-plattformen.

Ett Novacura Flow-system består av flera komponenter:

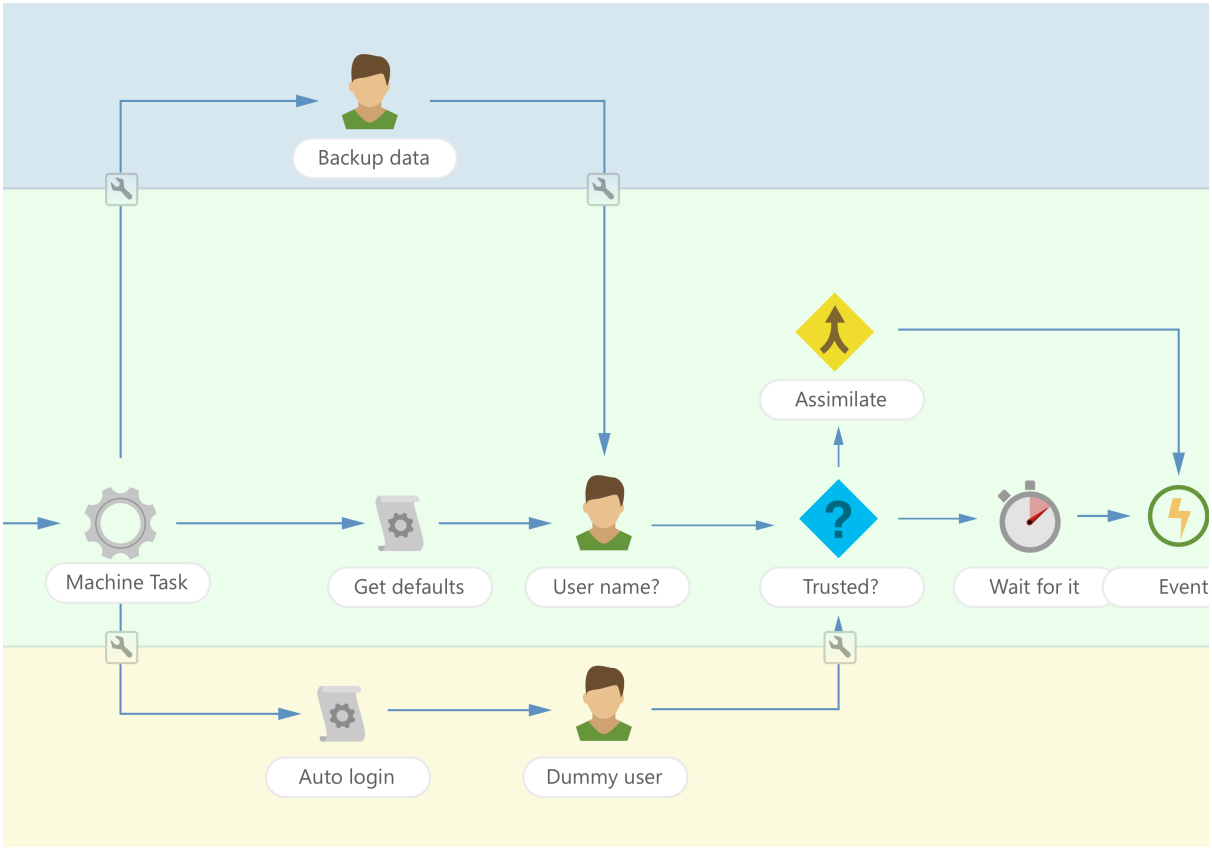
- En applikation för Windows kallad Novacura Flow 6 Studio, skriven i C#.
- Processer som man skapar grafiskt och konfigurerar i Flow Studio.
- En webbapplikation som utför den process man skapat, och som kompileras i AngularJS av Studio-applikationen.
- Ett antal konnektorer som kan kommunicera med olika affärssystem och databaser som IFS, SAP, IBM Maximo, Oracle, MySQL, och fler.
- En Flow Server som kör webbapplikationerna och konnektorerna.
- Klienter som kan använda webbapplikationerna via webbläsare eller Novacuras mobila applikationer för iOS, Android och Windows CE.



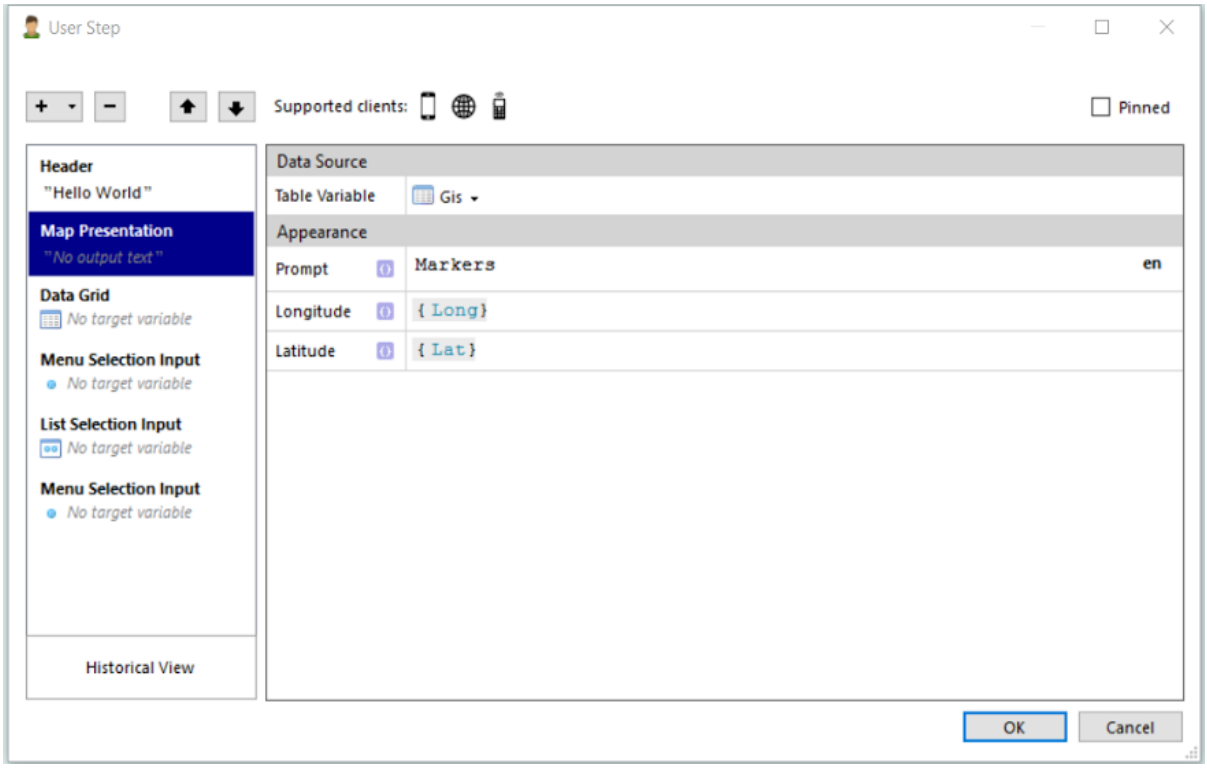
Figur 4: Bilden visar en illustration av informationsflödet mellan lagren i Novacura Flow. Konnektorerna programmeras med FlowScript för att fylla tabellen i server-lagret och används sedan som datakälla i "MapPresentation" (se Figur 5). I webblagret skapas sedan två objektstyper som adderar data till kartan.

När man skapar en Flow-process i Flow Studio länkar man ihop olika steg som kan vara till exempel användarsteg där användaren kan göra val och få data presenterad, eller maskinsteg som utför beräkningar på servernivå. Varje steg kan importera och exportera data från andra steg som ligger tidigare i flödet – exempelvis kan det användaren skriver in i en textruta i ett steg visas igen i ett senare steg, och en konnektor kan hämta data och spara i en tabell som sedan listas. Dessa steg byggs i sin tur upp av diverse komponenter, som exempelvis en listpresentation, databaskopplingar, textinmatningsfält och mycket annat. I denna existerande arkitektur blir alltså vårt mål att programmera en ny "kartkomponent" i C# för Flow Studio som kan konfigureras, och att den komponenten sedan vid kompilering producerar ett segment AngularJS-kod som kan visa upp en karta med de funktioner som krävs i applikationen.

Resultat från uppstartsfasen: Förståelse av Novacura Flow-plattformen; Arbete med kunskapsprodukten som presenterades i 5.3 och 5.4.



Figur 5: Exempel på ett processflöde i Novacura Flow 6 Studio. Ikonerna indikerar olika sorters steg i flödet, där exempelvis ett huvud är ett användarsteg. Färgerna är olika processroller som representeras grafiskt med s.k. ”swimlanes”.



Figur 6: Bilden visar konfigurationsrutan för ett användarsteg (”User Step”) i Flow Studio 6. Den vänstra kolumnen visar de komponenter som ingår i användarsteget. I den högra delen ser man de detaljer som kan konfigureras i kartkomponenten (kallad ”Map Presentation” i Flow Studio), vilket i denna tidiga prototyp inkluderar från vilken tabell man hämtar data, och från vilka kolumner i den tabellen som komponenten ska hämta latitud och longitud.

5.5.2 Utveckling av Prototyp 1

Efter att vi fått hjälp med att skapa grunden till en ny Flow-komponent började vi utforma komponenten efter den prioriterade kravlistan. Vi lade till möjligheten att koppla en tabell med data till kartkomponenten, och låta komponenten läsa ut vilka delar av den tabellen som var positionsdata (latitud och longitud) respektive övriga data som ska visas i kartan (popup-bubblans innehåll). Med JavaScript och AngularJS skapade vi en webbkarta som använder Leaflet-biblioteket och kunde hämta ut data ur den kopplade tabellen för att visa dessa punkter som markörer på kartan. Vi skapade en exempelprocess i Flow Studio 6 som visade att kartkomponenten fungerade. Dock fick vi lov att använda hårdkodade data i tabellen, eftersom Novacuras REST-konnektor fortfarande var under utveckling.

Resultat från utvecklingen av Prototyp 1: Insikt i Novacura Flows kodbas och uppbyggnad; Fungerande karta med markörer och databaskoppling.

5.5.3 Utveckling av Prototyp 2

Efter denna första prototyp med en fungerande datakoppling till Flow fokuserade vi på att utveckla själva kartans funktioner mer. Genom att införliva olika plug-ins till Leaflet kunde vi lägga till en funktion så att kartan hittar användarens position och ställer om kartvyn efter detta

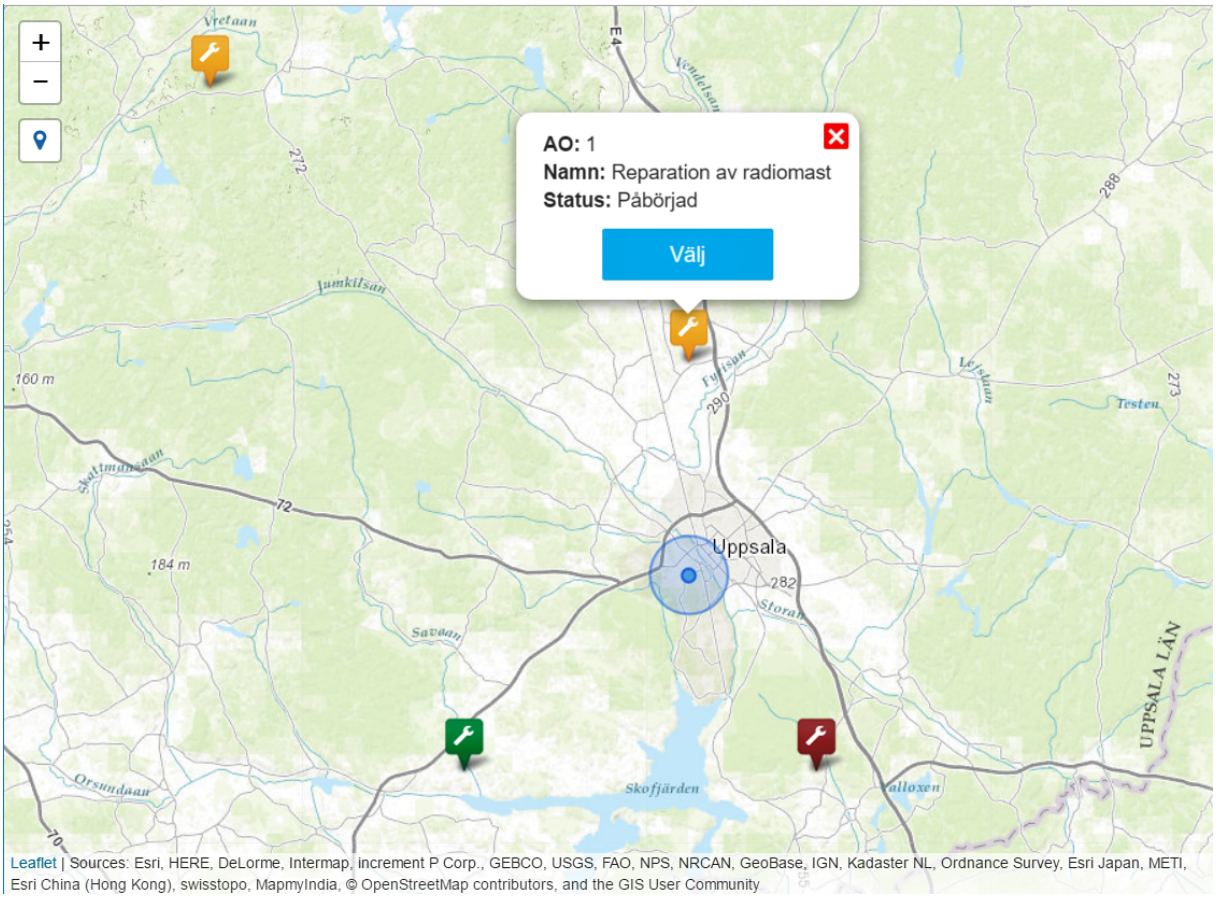
(Leaflet Locatecontrol[3]), förbättra variationen av markörer (Leaflet ExtraMarkers[4]) och klustrera markörer (Leaflet Markercluster[5]). Vid utvecklingen av denna del av kartkomponenten övergav vi tillfälligt Microsoft Visual Studio som vi tidigare använt som utvecklingsmiljö, både för att lättare kunna arbeta parallellt och eftersom vi bara hade koden till Novacura Flow Studio på en dator och inte kunde använda versionshantering. Detta innebar att utvecklingen gick snabbare, men samtidigt förlorade vi databaskopplingen som vår första prototyp hade, så vi fick lov att skapa nya kartlager med markörer direkt i Javascript-koden för att använda som exempel. I samband med detta tänkte vi över datastrukturerna för den inkommande informationen och utvecklade en enkel modell för att effektivt kunna behandla data från de geografiska punkterna och presentera i en popup-bubbla. I denna popup skapade vi också en metod för att välja en punkt och spara dess radindex till en variabel. Den ska sedan kunna kommuniceras till Flow-Server och låta användaren gå vidare i flödet för att exempelvis kunna se mer information om den valda arbetsordern och ändra dess status eller liknande. Detta blev vår andra prototyp, som vi skickade vidare till Novacuras utvecklingsavdelning.

Resultat från utvecklingen av Prototyp 2: Avancerade kartfunktioner som uppfyller fler av Kundens krav; Grundläggande datastruktur i kartan som Flow kan utnyttja.

[3] https://github.com/domoritz/leaflet-locatecontrol

[4] https://github.com/coryasilva/Leaflet.ExtraMarkers

[5] https://github.com/Leaflet/Leaflet.markercluster

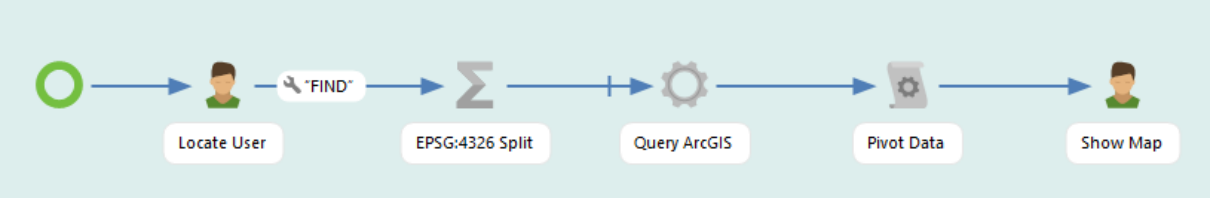


Figur 7: Exempelbild från prototypen som utvecklats. Utseendet på markörer och kartan beror på parametrar ur affärssystemet och GIS-databasen. Markörfärgen i detta exempel beror på status hos AO.

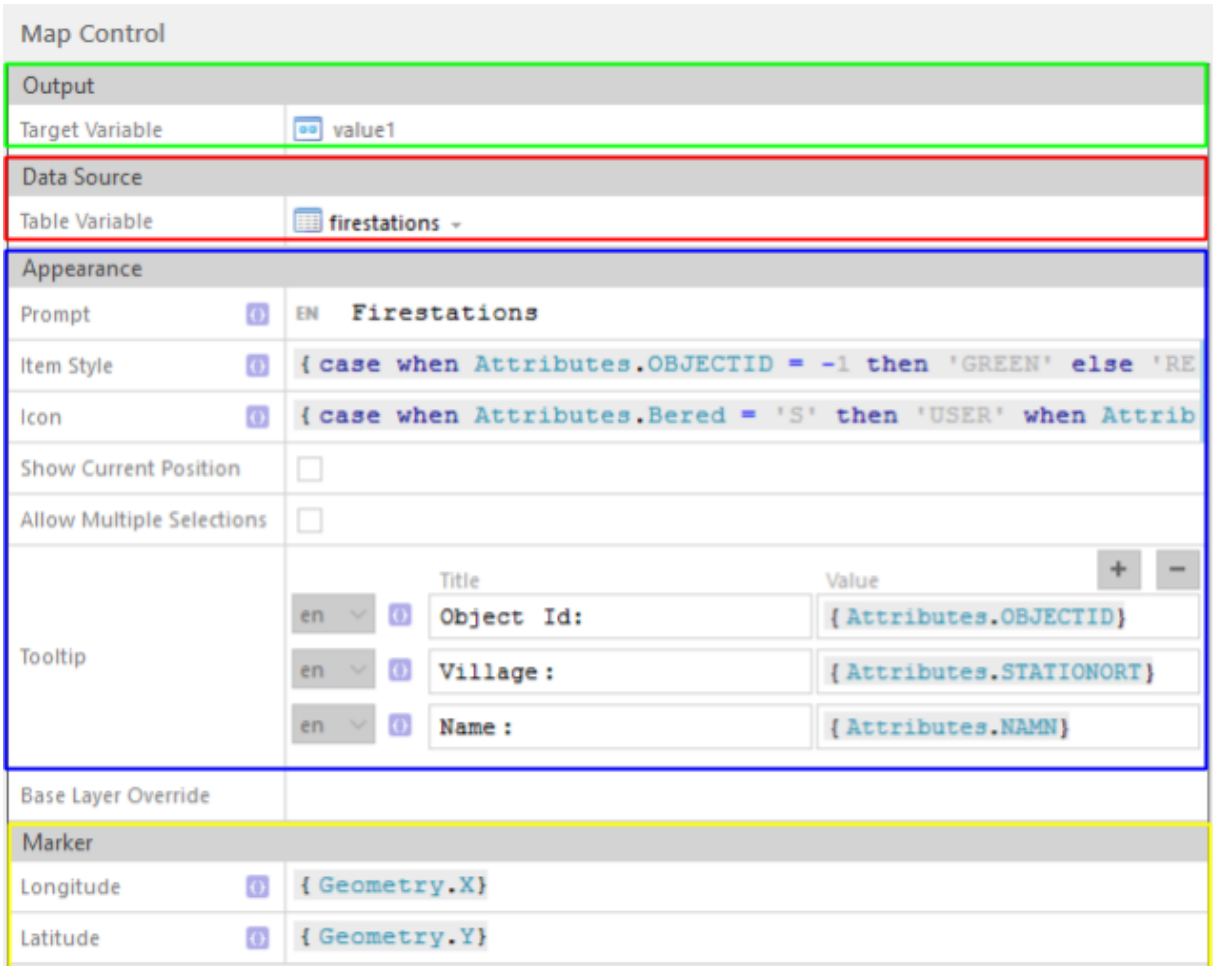
5.5.4 Utveckling av Prototyp 3

Nu fanns även REST-konnektorn i testbart skick hos Novacura. Problemet var att denna låg i en betaversion av Flow Studio 6.2, och all vår kod fanns i vår egen kopia av Flow Studio 6.1. Eftersom vi inte hade haft möjlighet att använda versionshantering, både av säkerhetsskäl och för att vi saknade utvecklarkonton på företaget, fick vi be Novacuras utvecklare ta den kod vi utvecklat och föra över till den nya versionen. I samband med det arbetet kunde de också lägga in ytterligare kontroller som vi planerat. Slutresultatet blev en tredje prototyp, som kunde använda REST-konnektorn för att hämta data från ArcGIS-lager och visa på en karta. Denna prototyp togs därefter med till Kunden för en demonstration.

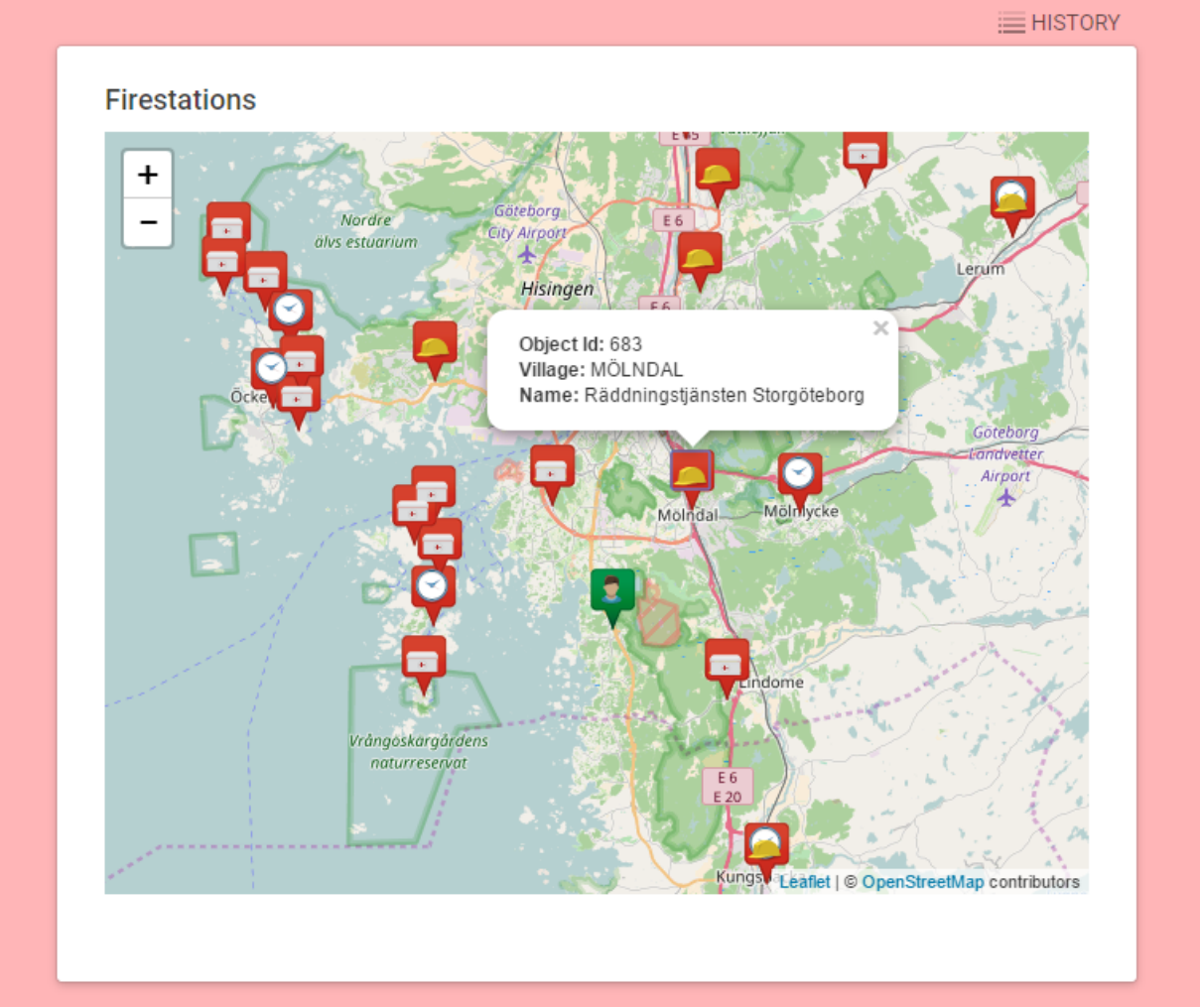
Resultat från utvecklingen av Prototyp 3: Flytt av funktionalitet från JavaScript-koden i Prototyp 2 till C# inuti Flow Studio; Koppling till en aktiv REST-konnektor.



Figur 8: Processflödet i demonstrationen av kartkomponenten. Flödet lokaliserar först användaren, kalkylerar om koordinaterna till latitud och longitud, skickar en förfrågan till ArcGIS och skapar en tabell av svaret som visas upp på en karta.



Figur 9: Konfiguration av kartkomponenten (användarsteg "Show Map" i Figur 8) i prototyp 3. Data tas från tabellen "firestations" som skapats tidigare i flödet (rött område). Ur tabellen hämtas variabler, som t.ex. data för markörer på positionen (gult område), eller data för utseende och popupinformation (blått område). Efter bearbetning av rådatan från konnektorn sparas de i "Target variable" (grönt område) vilken därefter är tillgänglig i kommande steg i flödet och som datakälla i applikationen.



Figur 10: Prototyp 3 demonstreras. På denna karta visas alla brandstationer (röda markörer) inom tre mil från användaren (grön markör).

5.6 Demonstration och utvärdering

Den tredje prototypen demonstrerades av oss och Novacuras utvecklare under ett möte på Kundens kontor, där även en konsult från ESRI var tillgänglig per telefon. De ansvariga hos Kunden var nöjda med utvecklingen, och såg fram emot att få testa kartfunktionen mot deras egna system när den är färdig för produktion i Flow 6.3, preliminärt i september 2017. Värdefull feedback framkom vilket kommer underlätta nästa fas av utvecklingsarbetet. Bland annat diskuterades hur man på ett effektivt sätt ska kunna söka information på existerande kartlager, hur REST-konnektorn ska kunna autentisera kopplingen till Kundens privata ArcGIS-server och andra detaljer.

Eftersom kartkomponenten inte kan sättas i händerna på verkliga användare eller Kundens projektansvariga ännu finns det ingen direkt möjlighet för dem att utvärdera resultaten av designen ännu. Som nämnts i 4.2 Metod kan man däremot utvärdera prototypens funktionalitet gentemot de kravspecifikationer som ställts upp för projektet, vilket vi gjort och sammanställt i 6.3, Analys av Artefakt.

6 Analys

Avsnittet granskar de resultat som uppnåtts, utgående från den metodik som använts och de forskningsresultat som eftersökts. Detta inkluderar en genomgång av kunskapsprodukten, artefakten och forskningsfrågorna.

6.1 Kriterier för designbaserad forskning

I teorikapitlet presenterades Hevners m.fl. (2004) sju riktlinjer för att utföra designbaserad forskning inom IS. Vår målsättning var att hålla oss till dessa riktlinjer under examensarbetet i så hög grad som möjligt.

I detta arbete har vi designat en artefakt (riktlinje 1), och visat på relevansen hos affärsproblemet som artefakten löser (riktlinje 2). Vi har använt en iterativ process vid utvecklingen av artefakten (riktlinje 6), och i denna process noggrant dokumenterat vårt arbete i loggbok, mötesanteckningar och kodkommentarer (riktlinje 5). En metodisk utvärdering av artefakten (riktlinje 3 och 5) har inte utförts eftersom detta skulle kräva en artefakt som är tillräckligt klar för att sättas i användarnas händer, men återkopplingen från Kunden efter demonstration av prototypen är positiv. Forskningsbidragen (riktlinje 4) argumenterar vi för nedan, vilket också utgör vår forskningskommunikation (riktlinje 7).

6.2 Kunskapsprodukt

Vid arbetets början var ett av de främsta målen för Novacura att få svar på hur man på bästa sätt fick olika mobila applikationer att kommunicera med varandra, eftersom det var så Kunden till en början hade beskrivit problemet. Det visade sig dock snart att en sådan kunskapsprodukt med rekommendationer inte var nödvändig eftersom sättet som Novacura Flow fungerar på gör en serverbaserad lösning till den självklara, och en teknikreview var inte längre nödvändig.

Istället fokuserade vi arbetet på utveckling av artefakten, och under den tiden utvärderade vi ett antal olika kartsystem och hur väl de passade för integration i Novacura Flow. Efter att ha vägt de fyra alternativen Leaflet, ArcGIS, Mapbox och Google Maps mot varandra valde vi att använda Leaflet till att börja med. Som ett starkt alternativ såg vi Mapbox, men eftersom det är en kommersiell lösning som kräver licenser ansåg vi att Novacura kan avvakta med att använda den tills man vet hur mycket pengar det är rimligt att satsa på kartkomponenten. Denna rekommendation till Novacura, och resonemanget som ledde till den, blir alltså vår slutgiltiga kunskapsprodukt.

6.3 Artefakt

Målsättningen med designen av en artefakt inom designbaserad forskning är att den ska lösa ett väldefinierat problem. I vårt fall löser den färdiga artefakten Kundens problem med att på ett smidigt sätt kunna leverera information om arbetsordrar till sina tekniker i fält. Informationen

fanns tillgänglig redan innan, men sättet teknikerna var nödgade att hämta den på var inte användarvänlig. Med hjälp av kartkomponenten behöver serviceteknikern endast använda en applikation som integrerar all nödvändig information från affärssystem och GIS-databasen som behövs för att utföra t.ex. ett reparationsarbete. Även om vår prototyp inte nått så långt att den är färdig för att levereras till användare än, så hävdar vi med stöd i de krav som uppfyllts att den är på god väg dit. I och med att Novacura kontinuerligt uppdaterar sin programvara med nya konnektorer och applikationskomponenter kommer kartkomponenten initialt att stödja de allra viktigaste funktionerna för att sedan utökas vid senare versioner av Flow. De krav vi har uppfyllt i prototypen synliggörs i Tabell 4.

Kundens användarkrav	Genomfört
Man ska kunna se på en karta var en Arbetsorder (AO) ska utföras.	Ja, komponenten skriver ut markeringar på kartan.
Man ska lätt kunna se vad AO:n har för prioritet och vad det är för typ av AO.	Ja, markeringar färgkodas beroende på status på AO t.ex.
Man ska kunna se vilken kund det är.	Delvis. Datan finns i IFS och kan skickas till kartapplikation och visas som popup-data.
Man ska se om det finns särskilda krav på AO:n, om specifika kompetenser behövs och hur många tekniker som behövs.	Delvis, Som ovan. Datan kan visas i popupbubblorna.
Det ska gå att se hur man kommer in på platsen (Access).	Delvis, som ovan. Datan kan visas i popupbubblorna.
Det ska finnas en kort beskrivning av ärendet.	Ja, visas i popupbubblorna.
AO:ns status ska gå att se och en notis ska sändas om det ändras från något håll.	Ja, funktionalitet som sköts i processflödet i Flow. Inget kartkomponenten hanterar.
Det ska framgå om AO:n kräver reservdelar och i så fall var det närmaste lagret är som har dessa delar.	Nej, datan går att visa i popupbubblorna men har inte designats specifikt för.
Det ska gå att filtrera efter kompetens, kund och liknande egenskaper hos AO.	Nej, däremot har Flow möjlighet att dela ut rollspecifika AO baserat på t.ex. kompetens. Konfigureras alltså i processflödet.
Man ska kunna se vilken Servicegrupp AO:n tillhör och vem som är områdesansvarig.	Delvis, har inte beaktats under utvecklingen men går att bifoga som data till markörerna.
Man ska kunna zooma på kartan.	Ja
Kartfunktionen ska ha en Gantt-vy som filtreras geografiskt efter vyn, och det ska gå att tilldela AO genom denna vy.	Nej
Kartan ska vara länkad till IFS så man kan se närmare detaljer om en AO via en knapp till och från IFS.	Ja, ytterligare data hämtas av maskinsteg i Flow och därigenom visas i applikationen men detta är utanför kartkomponenten
Kartan ska automatiskt uppdateras när nya AO läggs till i IFS.	Delvis, kartan uppdateras varje gång ett flöde körs och kartan instansieras. Uppdateras inte under användande.
Kartan ska visa om det föreligger kraftigt väder som stör verksamheten i ett område, typ åska eller storm.	Nej, men det går att visa ”Feature layers”. Men ytterligare utveckling krävs för att möta detta krav helt.
Kartan ska visa om AO ligger i ett farligt område, som t.ex. ett militärt övningsfält.	Ja, om geografiska data finns i ett ”Feature layer” så går det att visa på kartan.
Vid flera AO på samma plats ska man kunna se hur många det är.	Delvis, visas som flera markörer på samma plats. Klustring har provats men finns inte med i prototypen.
Det ska vara enkelt att tilldela AO samt göra statusändringar vid hantering av en AO.	Ja, detta sköts i ytterligare användarsteg i applikationen och konfigureras i Flow-processen.

Tabell 4: Visar kravlistan och vilken funktionalitet vi implementerat i prototypen.

Denna utvärdering av prototypens funktionalitet kan ses som en fullgod utvärdering av artefakten i avsaknad av verklig användartestning. På Kundens kravlista har vi tillgodosett alla de mest grundläggande kraven och lagt grunden för att en stor del av de andra kraven också ska gå att möta med rätt databaskoppling. Enligt denna utvärdering bör alltså prototypen nå över gränsen för en minsta brukbar produkt (Minimal Viable Product) när den väl finns tillgänglig för Kunden.

6.4 Forskningsfrågor

6.4.1 Har vi, genom att designa en kartkomponent, löst Kundens problem med anpassning av deras informationsinfrastruktur?

Anpassningsproblemen av Kundens II beror till stor del på att den tekniska lösning IFS erbjuder inte är tillräckligt flexibel. Genom att addera Novacura Flow-lagret till befintlig II möjliggörs hantering av den dynamiska komplexiteten inom Kundens II, samtidigt som man behåller den befintliga lösningen i underliggande lager. Liknande utmaningar med komplexitet möter givetvis även Novacura Flow. Utvecklingen av kartkomponenten och REST-konnektorn medför att Flow kan användas i ytterligare processer, d.v.s. hantera större komplexitet.

Den prototyp som vi har tagit fram är inte helt färdigutvecklad. Däremot finns ett antal grundläggande funktioner färdiga. Även om den ännu inte är fullt implementerad är de ansvariga hos Kunden nöjda med framstegen hittills. Enligt vår kognitiva utvärdering av hur artefakten når upp till de specificerade kraven kommer kartkomponenten, när den är färdigutvecklad, möjliggöra den systemanpassning Kunden efterfrågar. Resultatet leder till att deras servicetekniker får all information samlad i en och samma applikation, vilket slutligen ger effektivare servicearbeten tack vare en bättre anpassad II.

6.4.2 Kan vårt fall ge en insikt i om Hanseth och Lyytinens designprinciper bör uppdateras, och i så fall hur?

Hanseth och Lyytinens designprinciper för informationsinfrastrukturer är i huvudsak tänkta att lösa två problem: först det de kallar ”*the bootstrap problem*”, d.v.s. problemet med att starta en ny infrastruktur från noll, och sedan ”*the adaptability problem*”, som är problematiken i att förändra och anpassa infrastrukturer efter hand. Det förstnämnda har vi inte behandlat, eftersom Kundens olika system kan ses som en redan existerande informationsinfrastruktur på organisationsnivå. Däremot visar detta fall ett exempel på hur anpassningsproblemet kan hanteras av en organisation. Genom att använda Novacura Flow som sammanhållande struktur kan Kunden binda samman sina olika system, i detta fall IFS och ArcGIS, och därmed skapa en anpassningsbar infrastruktur som kan förändras utifrån tidigare oförutsedda användarkrav. Flow och liknande programvarulösningar inom området BPMS (Business Process Management Suite) som Appian, Pega, K2 och Bizagi kan alltså utnyttjas för att binda samman infrastrukturer som de *gateways* Hanseth och Lyytinen diskuterar, men bidrar också med egen funktionalitet. På detta sätt kan de bidra till att följa den femte designprincipen ”Modularize the II” bakom kulisserna.

Kundens användarfall visar dock värdet i att skapa integrerade lösningar för slutanvändaren, vilket Hanseth och Lyytinens designprinciper inte förordar, vilket de nämner i sin 14:e design-regel: ”D[esign]R[ule]14: Promote partly overlapping IT capabilities instead of all-inclusive ones.” (Hanseth & Lyytinen, 2010, s. 9). Snarare vänder de sig mot en ”allt-i-ett”-lösning, även om de då främst diskuterar själva infrastrukturen och inte enstaka användares situation. Syftet med denna designprincip ska vara att tillåta förändring inom II, vilket de anser underlättas om olika IT-system är löst kopplade. Vikten av denna princip minskar dock i mindre II som en företagstäckande, där behovet av innovation kanske kan täckas av de möjligheter till anpassning som en BPRS som Novacura Flow kan ge. Den begränsning som BPR-systemet innebär täcks då upp av nyttan med större integration mellan olika system.

Vidare så har designprinciperna under utvecklingsarbetet i många fall varit triviala och närmast självklara, både från vår sida och Novacuras. Utifrån resultaten kan endast konstateras att så är fallet, orsaken till detta har inte studerats. Att döma av den litteratur vi tagit del av där fall av II-utveckling studerats har man likaledes inte följt designprinciperna aktivt utan dessa har kunnat appliceras på fallen i efterhand, vilket antyder att de är generiska eller allmänt underförstådda inom branschen.

7 Slutsats

Under detta examensarbete har uppdraget varit att stödja Novacura i vidareutvecklingen av deras processhanteringsplattform Novacura Flow. Flow syftar till att förbättra företags affärsprocesser genom att binda samman dem med olika konnektorer i ett processflöde och därigenom skapa skräddarsydda applikationer. Novacura Flow bildar tillsammans med integrerade system och användare en informationsinfrastruktur (II) i linje med Hanseth och Lyytinens (2004) definition. En II är inte statisk utan utvecklas ständigt genom teknikutveckling och nya användarsituationer. Detta kräver anpassning av II:en vilket Hanseth och Lyytinen definierar som ”*the adaptability problem*”. Syftet med uppsatsen har därför varit att utgå ifrån uppdraget vi erhållit av Novacura. Utvecklingsarbetet nyttjades sedan som ett mini-fall för att undersöka hur designutmaningar kan hanteras i en modern II samt hur anpassning av affärssystem kan ske i praktiken med hjälp av tredjepartsmjukvara.

Uppdraget vi gjort tillsammans med Novacura har lett till utvecklingen av en artefakt som, enligt vår analys, med någon ytterligare iteration kan lösa den ursprungliga problematiken Kunden hade. Under denna utveckling har vi sett paralleller mellan hur Novacura binder samman olika system – i detta fall IFS och ArcGIS – och de II som Hanseth och Lyytinen diskuterar. Eftersom de anpassnings- och användbarhetsproblem som existerar inom II även återfinns i stora affärssystem kan detta relateras till problematik vi diskuterat i avsnitt 2.

Vi vill hävda att Novacura Flow – och kanske även andra BPMS-system som vi ej studerat – i detta agerar som en plattform för att skapa och binda samman nya infrastrukturer. Det är näraliggande att kalla det en stödjande infrastruktur (Support II), men med dessa menar man oftast grundläggande transportprotokoll som TCP/IP och liknande lösningar som Enterprise Service Bus (ESB, programvara som hanterar kommunikation mellan exempelvis affärssystem) eller stödjande tjänster som DNS. Med sin möjlighet till anpassning och individualisering löser Novacura Flow anpassningsproblemet i Kundens II, vilket försäkrar att den II som skapats kan fortsätta fungera och utvecklas i framtiden även om användarkraven förändras. Vi föreslår en gängse term för dessa system och har valt att kalla dem, tillsammans med Novacura Flow, för **Information Infrastructure Enablers**, eftersom de på ett enkelt sätt möjliggör skapandet och hanterandet av II:er.

8 Diskussion

Konflikten mellan att å ena sidan utföra ett bra arbete för Novacura, som beställt arbetet, och att å andra sidan besvara en forskningsfråga har varit en konstant svårighet i detta projekt. På grund av ändrade prioriteringar från Novacuras sida har vi under arbetets gång fått omvärdera forskningen och ändra vår frågeställning. Tyvärr har vi då inte har haft möjlighet att utforska andra vägar att samla in och analysera data samtidigt som vi utfört Novacuras uppdrag.

Hanseth och Lyytinens designprinciper för informationsinfrastrukturer publicerades första gången 2004, vilket nu är 13 år sedan. Under den tiden har givetvis mycket hänt inom IS-världen, inte minst utvecklingen av smarta telefoner som tog verklig fart med lanseringen av iPhone 2007 och den kraftiga framväxten av s.k. molntjänster och webbapplikationer som delvis fått bränsle av just de smarta telefonernas fokus på applikationer. En naturlig fråga är om dessa designprinciper fortfarande är relevanta idag, eller om de är i behov av uppdatering?

Som vi nämner i vår analys är många av de principer som Hanseth och Lyytinen beskriver närmast självklarheter. Som exempel kan man konstatera att med en allt mer uppkopplad värld har interoperabillitet mellan system blivit allt viktigare och det är av den anledningen givet att utvecklare bygger in stöd för standarder i sina system. Den allt starkare rörelsen kring fri källkod (open source) och möjligheten att använda fria API:er eller att återanvända kod från tjänster som Github och Stack Overflow bidrar sannolikt också till att användandet av öppna standarder är allt vanligare. Med det sagt är det inget fel med designprinciper som känns självklara, men de ger ingen vägledning för framtida infrastrukturutveckling.

En frågeställning som dykt upp under arbetet har varit om utvecklingen av exempelvis molntjänster och stora sociala nätverk medför att de II som IT-tjänster bygger på alltmer tas för givet? Paralleller kan dras till affärssystemen som behandlats i uppsatsen där de erbjuder ett anpassningsbart system där II:en redan är etablerad. Är detta början på en ny era?

Vi ställer oss också frågande till om Hanseth och Lyytinens råd att undvika allt-i-ett-lösningar verkligen har slutanvändaren i åtanke. En alternativ lösning på Kundens problem hade ju varit att inte använda Novacura Flow utan att koppla IFS mobila applikationer till ArcGIS applikationer eller liknande, men även om det också hade blivit en II hade den inte varit lika användarvänlig. Både popularitet hos smarta telefoner och framväxten av Facebook från att vara en fristående hemsida till en av jordens största II:er (Brænden m.fl., 2015) signalerar att användare föredrar integrerade lösningar så länge de ger en bättre upplevelse (Harmon & Mazmanian, 2013).

Dessa indicier pekar på att vidare forskning rörande en uppdatering av designprinciperna för informationsinfrastrukturer är av intresse.

9 Källhänvisningar

Adam, R., Kotze, P., & Van der Merwe, A. (2011). Acceptance of enterprise resource planning systems by small manufacturing Enterprises. I *Proceedings of the 13th International Conference on Enterprise Information Systems* (s. 229–238). Beijing, China. Hämtad från http://ai2-s2-pdfs.s3.amazonaws.com/53cc/b99452403337e38470061917f105a04-e4672.pdf

Al-Baik, O., & Miller, J. (2015). The kanban approach, between agility and leanness: a systematic review. *Empirical Software Engineering*, *20*(6), 1861–1897. https://doi.org/10.1007/s10664-014-9340-x

Alban, S. (2015, mars 2). Independent Report Highlights Esri as Leader in Global GIS Market. Hämtad 26 maj 2017, från http://www.esri.com/esri-news/releases/15-1qtr/independent-report-highlights-esri-as-leader-in-global-gis-market

Basoglu, N., Daim, T., & Kerimoglu, O. (2007). Organizational adoption of enterprise resource planning systems: A conceptual framework. *The Journal of High Technology Management Research*, *18*(1), 73–97. https://doi.org/10.1016/j.hitech.2007.03.005

Bowker, G. C. (1996). The history of information infrastructures: The case of the international classification of diseases. *Information Processing & Management*, *32*(1), 49–61. https://doi.org/10.1016/0306-4573(95)00049-M

Bray, T. (2014). The JavaScript Object Notation (JSON) Data Interchange Format. Hämtad 26 maj 2017, från https://tools.ietf.org/html/rfc7159

Brehm, L., Heinzl, A., & Markus, M. L. (2001). Tailoring ERP systems: a spectrum of choices and their implications. I *Proceedings of the 34th Annual Hawaii International Conference on System Sciences* (s. 9 pp.-). https://doi.org/10.1109/HICSS.2001.927130

Broadbent, M., & Weill, P. (1997). Management by maxim: How business and IT managers can create IT infrastructures. *Sloan Management Review*, *38*(3), 77–92.

Brænden, K., Hagen, N. A., & Ommundsen, S. O. (2015). *Facebook - From Webpage to an Information Infrastructure*. Oslo: Oslo University. Hämtad från http://www.uio.no/studier/emner/matnat/ifi/INF5210/h15/project-reports/facebook.pdf

Business process management. (2017, maj 31). I *Wikipedia*. Hämtad från https://en.wikipedia.org/w/index.php?title=Business_process_management&oldid=783103059

Bygstad, B. (2010). Generative mechanisms for innovation in information infrastructures. *Information and Organization*, *20*(3–4), 156–168. https://doi.org/10.1016/j.infoandorg.2010.07.001

Ciborra, C. U., & Hanseth, O. (1998). From tool to Gestell: Agendas for managing the information infrastructure. *Information Technology & People*, *11*(4), 305–327. https://doi.org/10.1108/09593849810246129

Crawford, T. (2016, september 27). Is the concept of enterprise lock in a red herring? Hämtad 11 maj 2017, från https://avoa.com/2016/09/27/is-the-concept-of-enterprise-lock-in-a-red-herring/

DeMers, M. N. (2017). GIS. I *Britannica Academic*. Hämtad från http://academic.eb.com/levels/collegiate/article/GIS/396653

Eriksson, O., & Goldkuhl, G. (2013). Preconditions for public sector e-infrastructure development. *Information and Organization*, *23*(3), 149–176. https://doi.org/10.1016/j.infoandorg.2013.04.001

Fielding, R. T., & Taylor, R. N. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

Fuller, R. B. (u.å.). Fuller on Design Science. Hämtad 11 maj 2017, från https://www.bfi.org/design-science/primer/fuller-design-science

Fuller, R. B., & McHale, J. (1964). World Design Science Decade. Hämtad 11 maj 2017, från https://www.bfi.org/design-science/primer/world-design-science-decade

Google Inc. (2017). FAQ | Google Maps APIs. Hämtad 26 maj 2017, från https://developers.google.com/maps/faq

Hanseth, O., & Lyytinen, K. (2004). Theorizing about the Design of Information Infrastructures: Design Kernel Theories and Principles. I *Sprouts: Working Papers on Information Environments, Systems and Organizations 4(Fall 2004): Article 1* (s. 1–35). W.

Hanseth, O., & Lyytinen, K. (2010). Design theory for dynamic complexity in information infrastructures: the case of building internet. *Journal of Information Technology*, *25*(1), 1–19. https://doi.org/10.1057/jit.2009.19

Harmon, E., & Mazmanian, M. (2013). Stories of the Smartphone in Everyday Discourse: Conflict, Tension & Instability. I *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (s. 1051–1060). New York, NY, USA: ACM. https://doi.org/10.1145/2470654.2466134

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. *MIS Quarterly*, *28*(1), 75–105.

Jensen, T. B. (2013). Design principles for achieving integrated healthcare information systems. *Health Informatics Journal*, *19*(1), 29–45. https://doi.org/10.1177/1460458212448890

Leaflet. (2017). Leaflet — an open-source JavaScript library for interactive maps. Hämtad 23 maj 2017, från http://leafletjs.com/

Mapbox, Inc. (2017a). How web maps work. Hämtad 23 maj 2017, från https://www.mapbox.com/help/how-web-maps-work/

Mapbox, Inc. (2017b). Terms of service. Hämtad 26 maj 2017, från https://www.mapbox.com/tos/

Miller, A. (2010, december 17). Under the hood of Google Maps 5.0 for Android. Hämtad 12 maj 2017, från https://googleblog.blogspot.com/2010/12/under-hood-of-google-maps-50-for.html

Novacura AB. (2015, mars 6). Vi levererar processförbättringar [Text]. Hämtad 11 maj 2017, från http://www.novacura.se/detta-kan-vi-gora-for-dig

Palmer, N. (2014, mars 26). What Is BPM? Hämtad 09 juni 2017, från https://bpm.com/what-is-bpm

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Seroter, R. (2016, juni 8). Everything Is "Lock-In": Focus on Switching Costs. Hämtad 11 maj 2017, från https://www.infoq.com/articles/avoiding-lockin-switching-costs

Simon, H. A. (1969). *The sciences of the artificial* (1st ed). Cambridge, Mass: MIT Press. Hämtad från https://market.android.com/details?id=book-l8thngEACAAJ

Star, S. L., & Ruhleder, K. (1996). Steps Toward an Ecology of Infrastructure: Design and Access for Large Information Spaces. *Information Systems Research*, *7*(1), 111–134. https://doi.org/10.1287/isre.7.1.111

Zwass, V. (2017). information system. I *Britannica Academic*. Hämtad från http://academic.eb.com/levels/collegiate/article/information-system/126502